The Transformer

NLP Week 7

Thanks to Dan Jurafsky for most of the slides this week!

What we will cover:

LANGUAGE MODELS

- **N-gram language models** — Probabilistic models of language in context
- **Embeddings** — vector semantics
- **Recurrent neural language models, LSTMs** — neural networks for language
- **Transformers** — large language model (LLM) architectures
- **Encoder models** — scoring tasks
- **Encoder-decoder models** — generative tasks
- **Finetuning and in-context learning** — current foundation model paradigm

What we will cover:

LANGUAGE MODELS

- **N-gram language models** — Probabilistic models of language in context
- **Embeddings** — vector semantics
- **Recurrent neural language models, LSTMs** — neural networks for language
- **Transformers** — large language model (LLM) architectures
- **Encoder models** — scoring tasks
- **Encoder-decoder models** — generative tasks
- **Finetuning and in-context learning** — current foundation model paradigm

What we will cover:

LANGUAGE MODELS

- **N-gram language models** — Probabilistic models of language in context
- **Embeddings** — vector semantics
- **Recurrent neural language models, LSTMs** — neural networks for language
- **Transformers** — large language model (LLM) architectures
- **Encoder models** — scoring tasks
- **Encoder-decoder models** — generative tasks
- **Finetuning and in-context learning** — current foundation model paradigm

Plan for today

1. Review of matrix multiplication
2. Simplified attention
3. Multi-head attention
4. Transformer blocks and the residual stream
5. Transformer inputs and position embeddings
6. Transformer ouputs and the language modeling head
7. *Group exercises*

Notation in the book

Quick recap on our notation and matrix-matrix and matrix-vector multiplication

- Let $\mathbf{W}$ denote an $d \times p$ matrix
- Let $\mathbf{X}$ denote an $n \times d$ matrix
- Let $\mathbf{x}$ denote a $1 \times d$ vector (row vector)
- $\mathbf{x}^\top$ is a $d \times 1$ vector (column vector)
- Note that: $(\mathbf{XW})^\top = \mathbf{W}^\top \mathbf{X}^\top$

We need to understand:

- $\mathbf{y} = \mathbf{xW}$
- $\mathbf{y}^\top = \mathbf{W}^\top \mathbf{x}^\top$
- $\mathbf{Y} = \mathbf{XW}$
- $\mathbf{Y}^\top = \mathbf{W}^\top \mathbf{X}^\top$

👉 will do this on the whiteboard …

The paper that started it all

Transformer: a specific kind of network architecture, like a fancier feedforward network, but based on attention

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

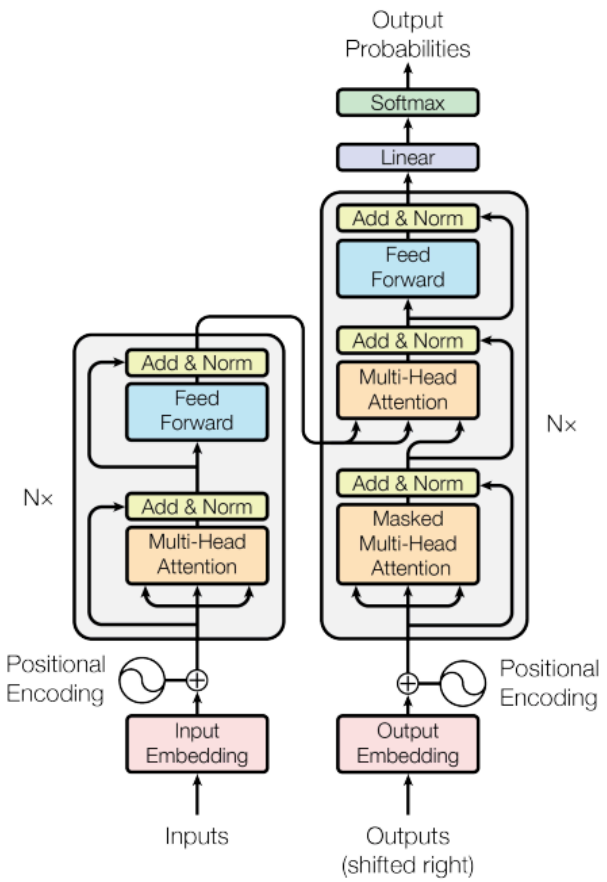
Llion Jones* Google Research llion@google.com

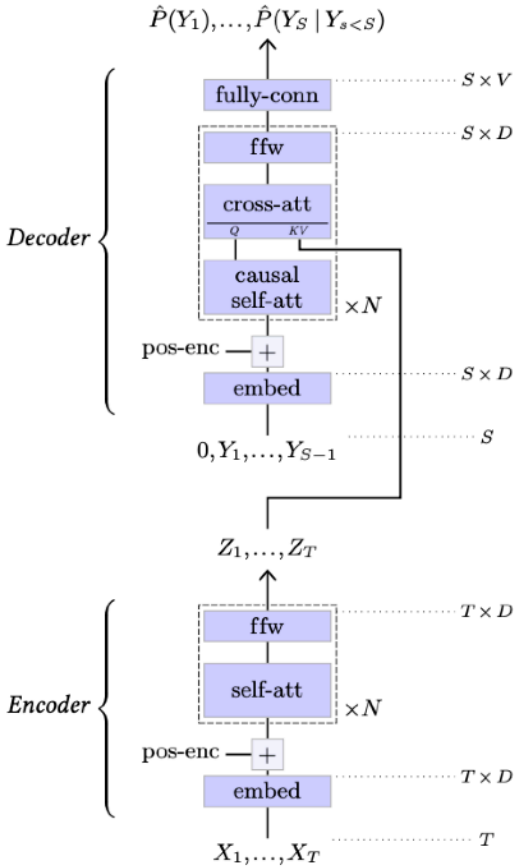
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

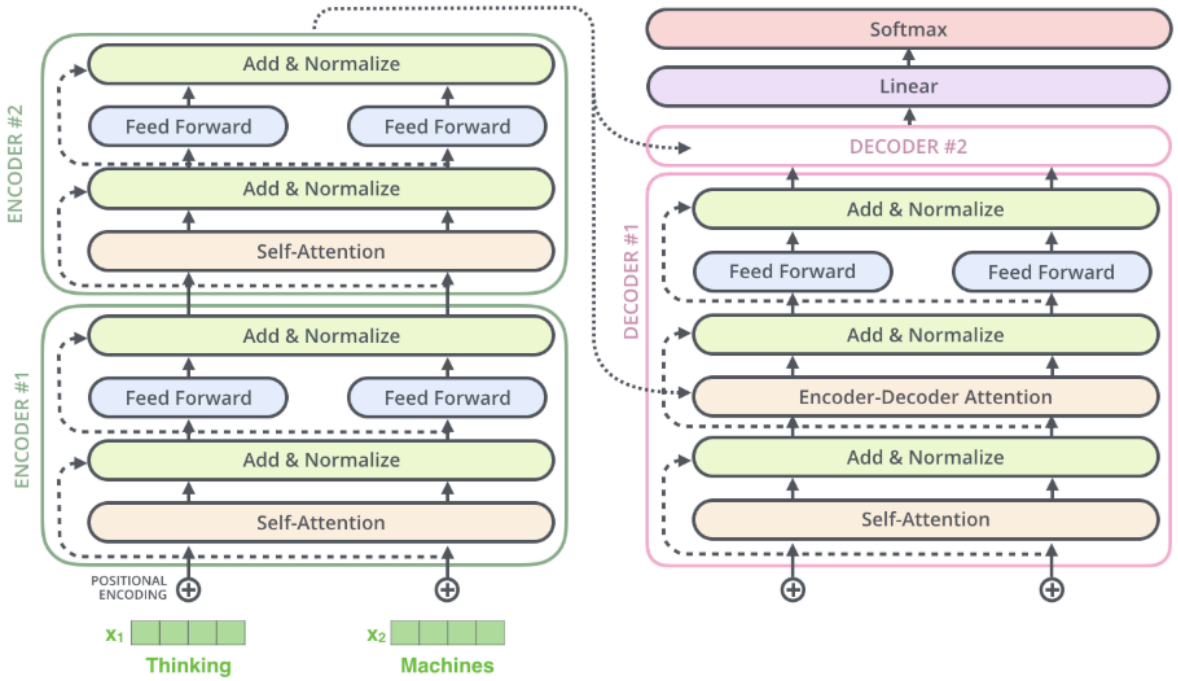
Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

What is a Transformer?





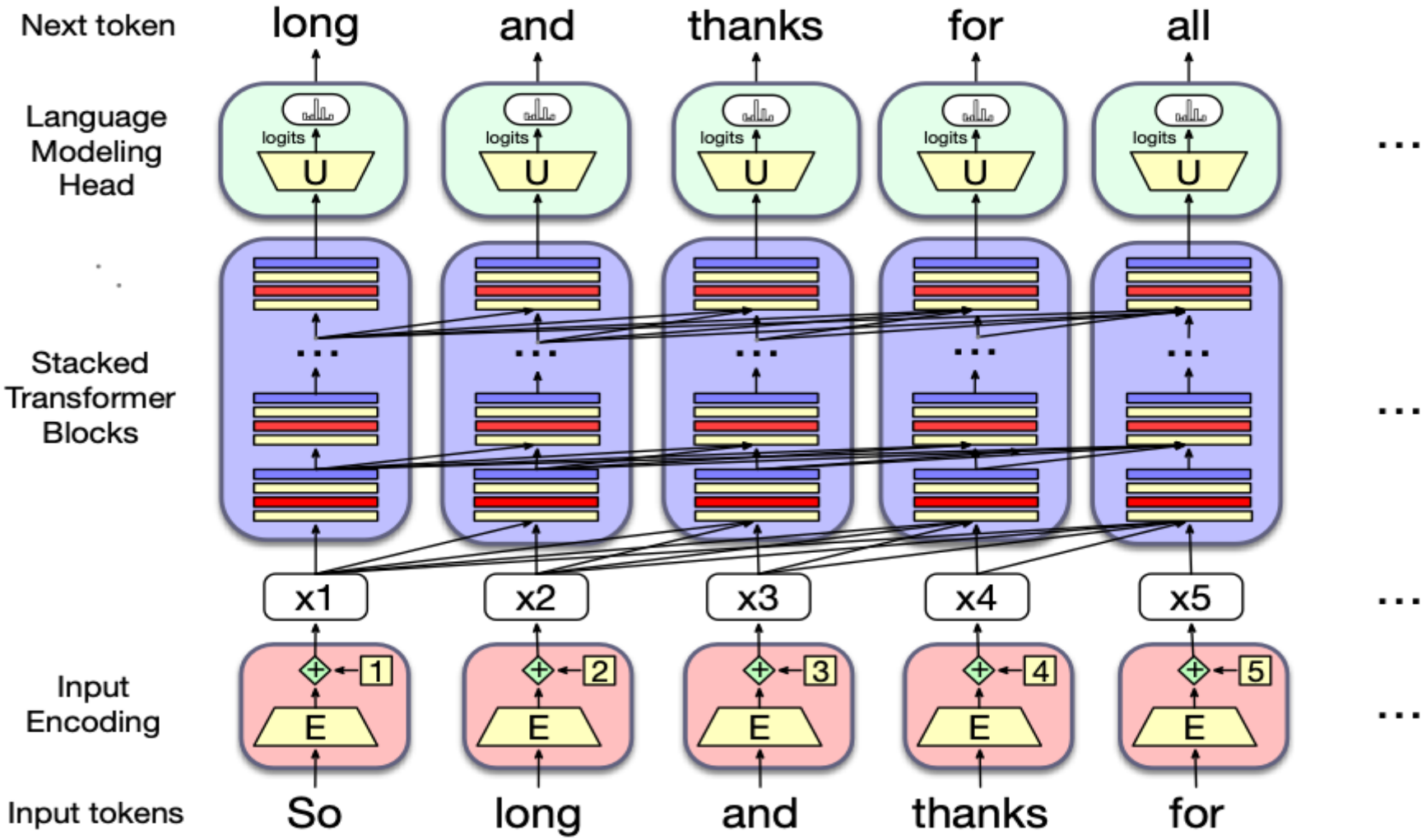


Vaswani et al. 2017 - Attention is all you need

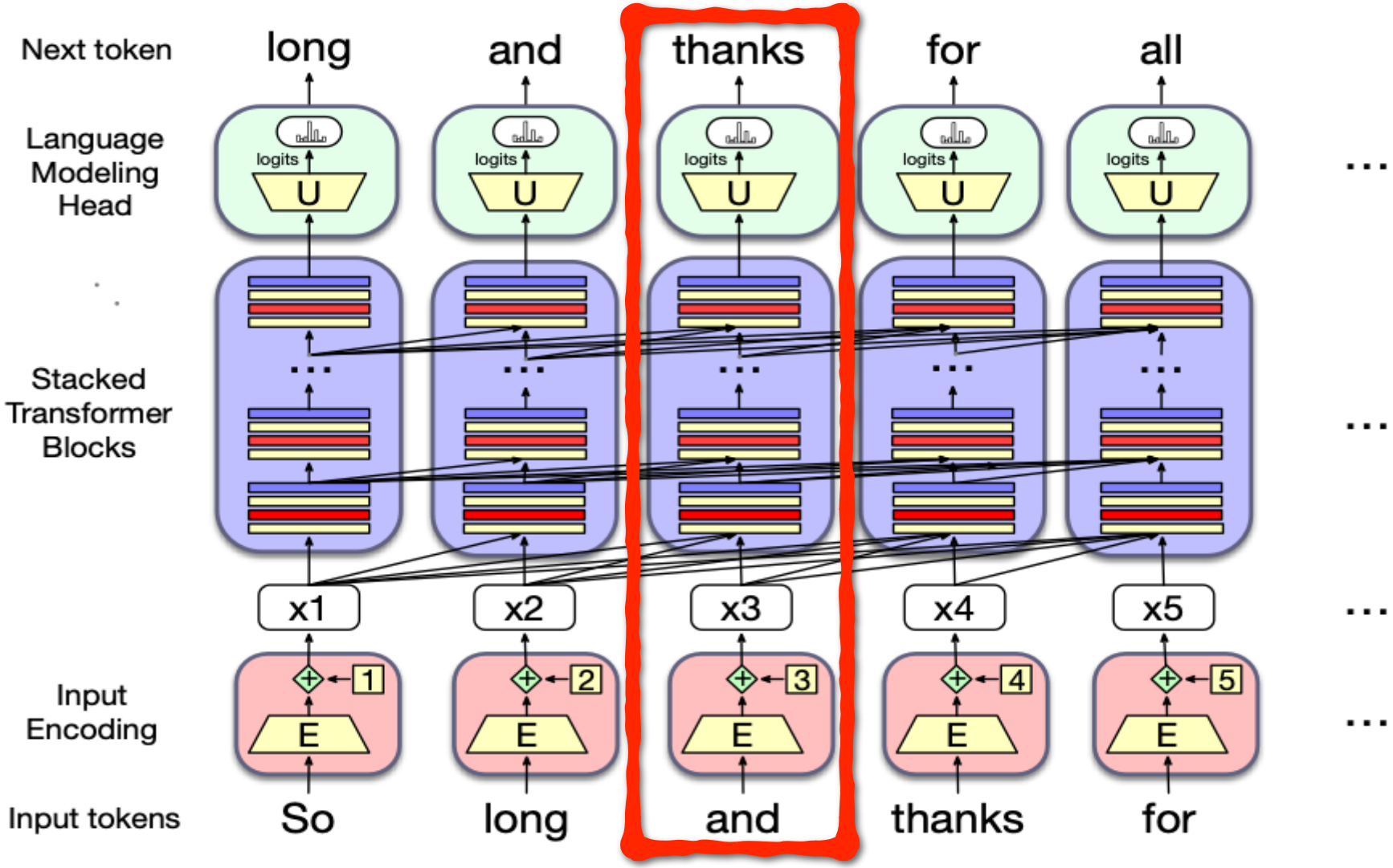
Fleuret et al. 2024 - The little book of DL

Alammar et al. 2018 - The Illustrated Transformer

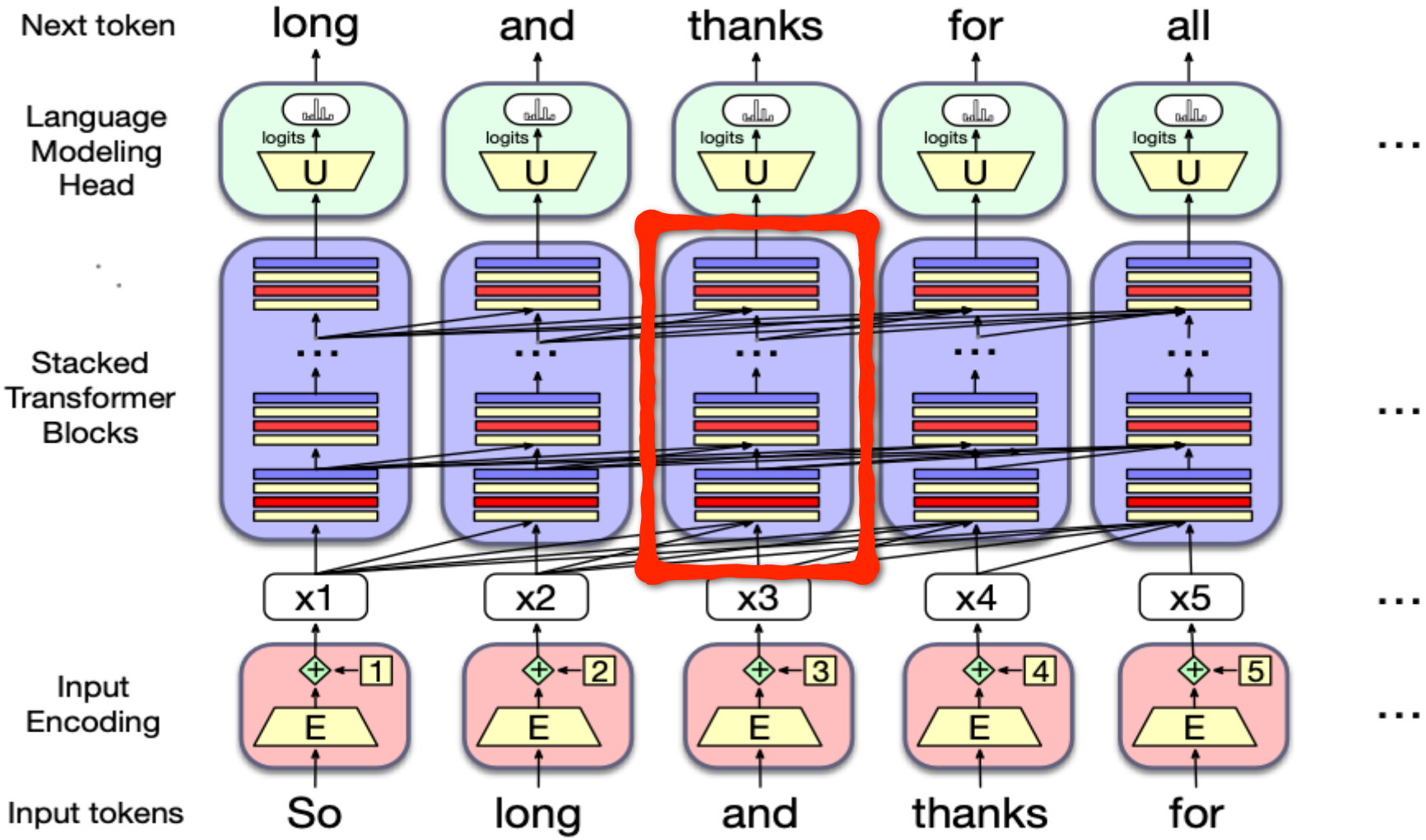
We will stick to the following illustration



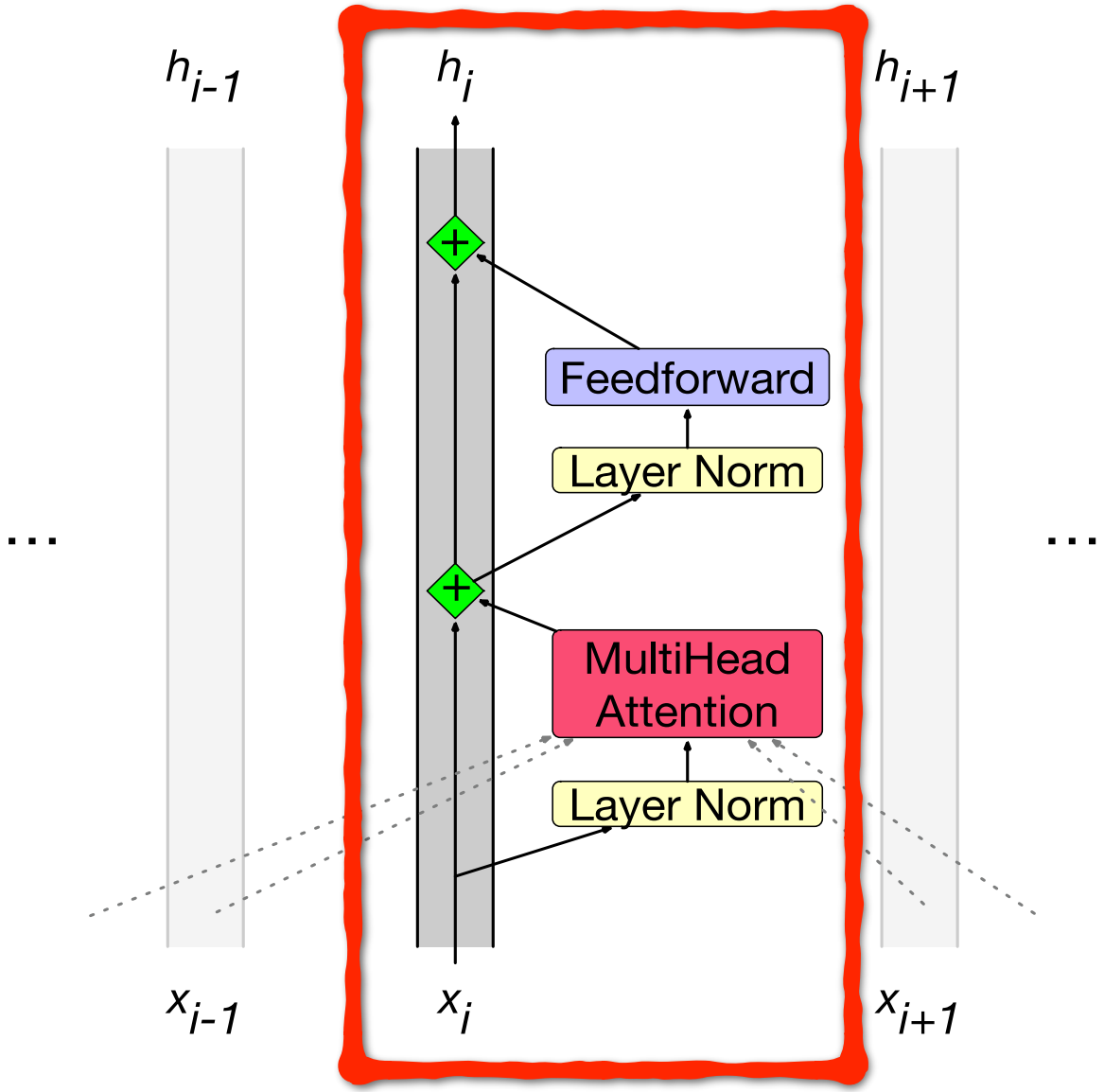
Let's consider what happens to the embeddings for one word



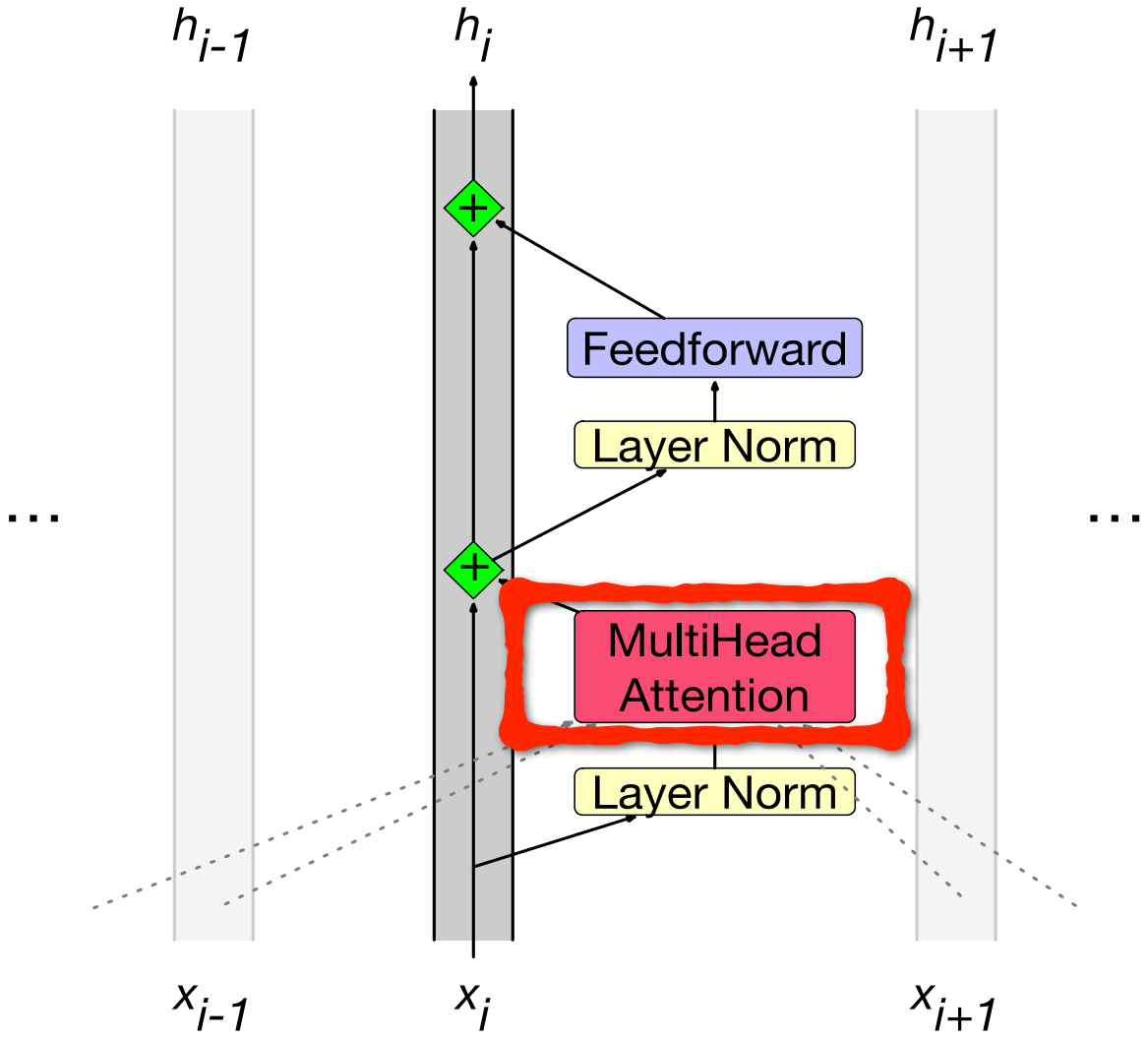
Transformer blocks



Zooming in : A transformer block



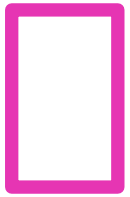
Zooming in : A transformer block - Attention head



Intuition of attention

- Build up the representation of a word by selectively integrating information from all the neighbouring words
- We say that a word "attends to" some neighbouring words more than others

Problem with static embeddings (word2vec)



Problem with static embeddings (word2vec)

They are static! The embedding for a word doesn't reflect how its meaning changes in context.

```
 The chicken didn't cross the road because it was too
tired
```

What is the meaning represented in the static embedding for "it"?

Contextual Embeddings

- Intuition: a representation of meaning of a word should be different in different contexts!
- **Contextual Embedding**: each word has a different vector that expresses different meanings depending on the surrounding words

Contextual Embeddings

- Intuition: a representation of meaning of a word should be different in different contexts!
- **Contextual Embedding**: each word has a different vector that expresses different meanings depending on the surrounding words
- How to compute contextual embeddings?
  - **Attention**

Contextual Embeddings

```
The chicken didn't cross the road because it
```

What should be the properties of "it"?

Contextual Embeddings

`The chicken didn't cross the road because it`

What should be the properties of "it"?

`The chicken didn't cross the road because it was too` **`tired`**
`The chicken didn't cross the road because it was too` **`wide`**

Contextual Embeddings

`The chicken didn't cross the road because it`

What should be the properties of "it"?

`The chicken didn't cross the road because it was too` **`tired`**
`The chicken didn't cross the road because it was too` **`wide`**

At this point in the sentence, it's probably referring to either the chicken or the street

Contextual Embeddings

`The chicken didn't cross the road because it`

What should be the properties of "it"?

`The chicken didn't cross the road because it was too` **`tired`**
`The chicken didn't cross the road because it was too` **`wide`**

At this point in the sentence, it's probably referring to either the chicken or the street

Contextual Embeddings

`The chicken didn't cross the road because it`

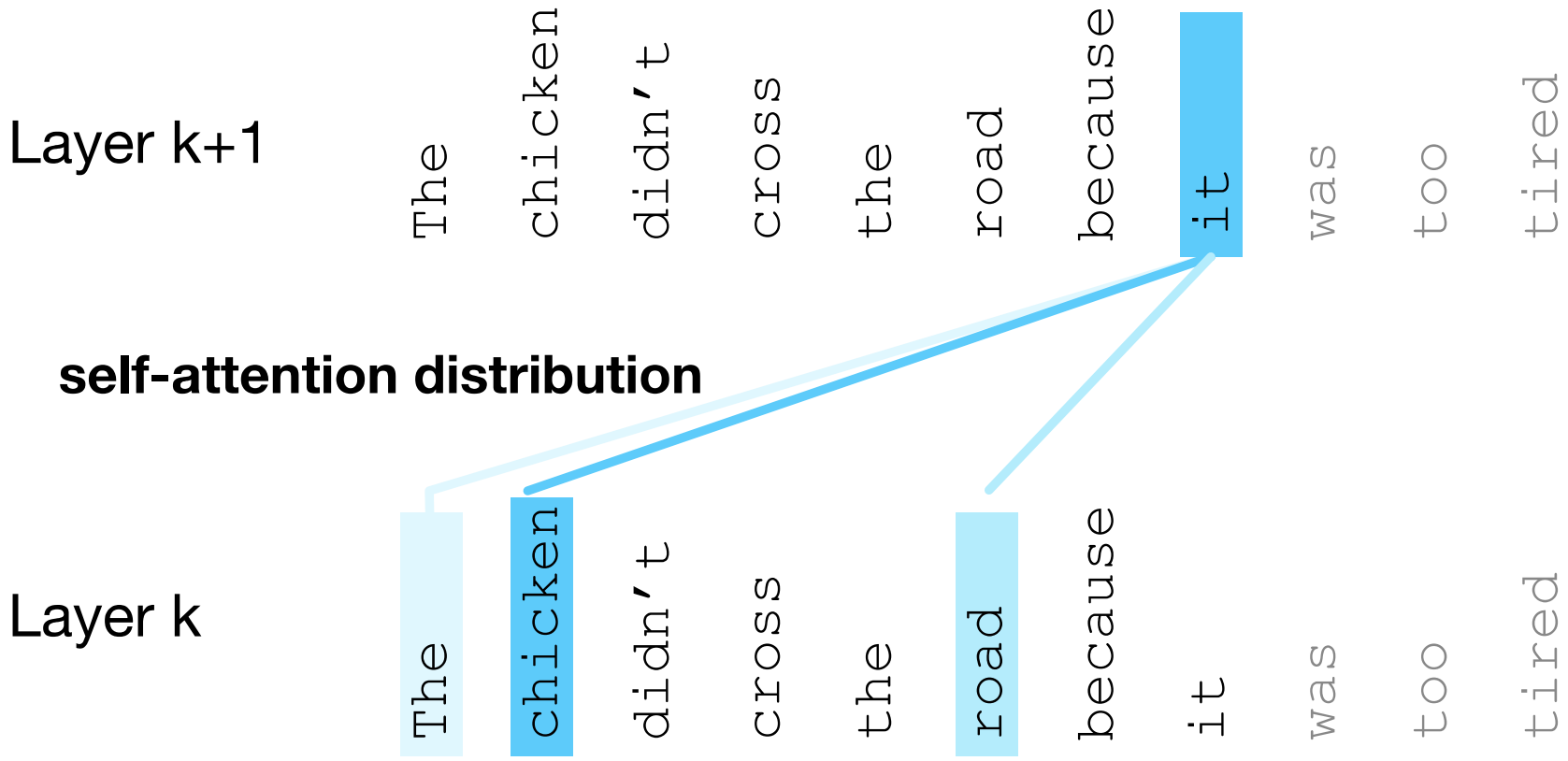
What should be the properties of "it"?

`The chicken didn't cross the road because it was too` **`tired`**
`The chicken didn't cross the road because it was too` **`wide`**

At this point in the sentence, it's probably referring to either the chicken or the street

Intuition of attention:

columns corresponding to input tokens



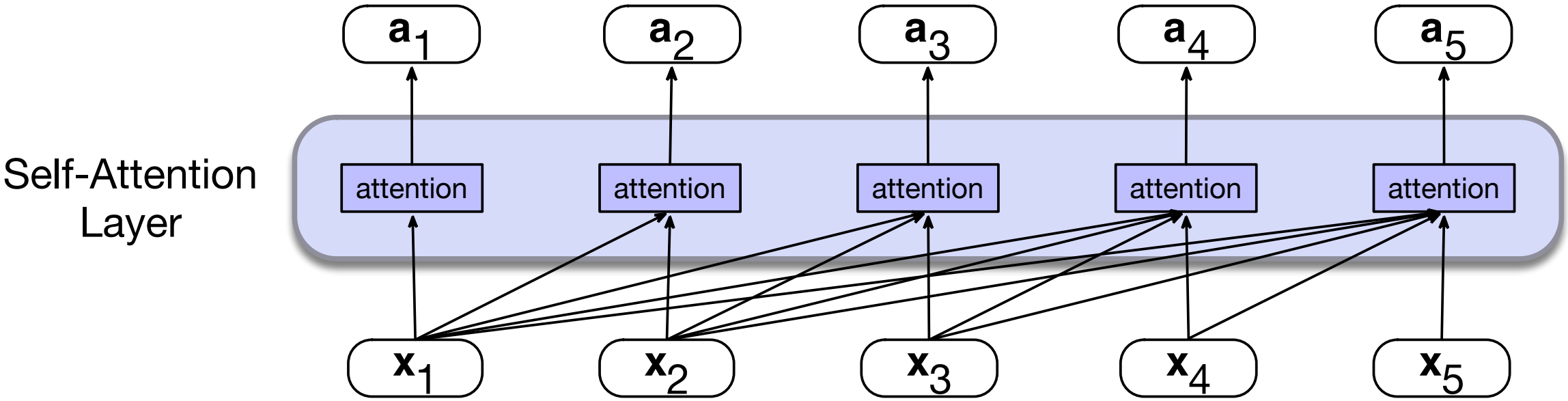
Attention definition

A mechanism for helping compute the embedding for a token by selectively attending to and integrating information from surrounding tokens (at the previous layer).

More formally: a method for doing a weighted sum of vectors.

$$\mathbf{a}^i = \sum_{j \leq i} \alpha_{ij} \cdot \mathbf{x}_j$$

Attention is left-to-right



Simplified version of attention: a sum of prior words weighted by their similarity with the current word

Given a sequence of token embeddings:

$$\mathbf{x}_1 \quad \mathbf{x}_2 \quad \mathbf{x}_3 \quad \mathbf{x}_4 \quad \mathbf{x}_5 \quad \mathbf{x}_6 \quad \mathbf{x}_7 \quad \mathbf{x}_i$$

Produce: $\mathbf{a}_i$ = a weighted sum of $\mathbf{x}_1$ through $\mathbf{x}_7$ (and $\mathbf{x}_i$), weighted by their similarity to $\mathbf{x}_i$

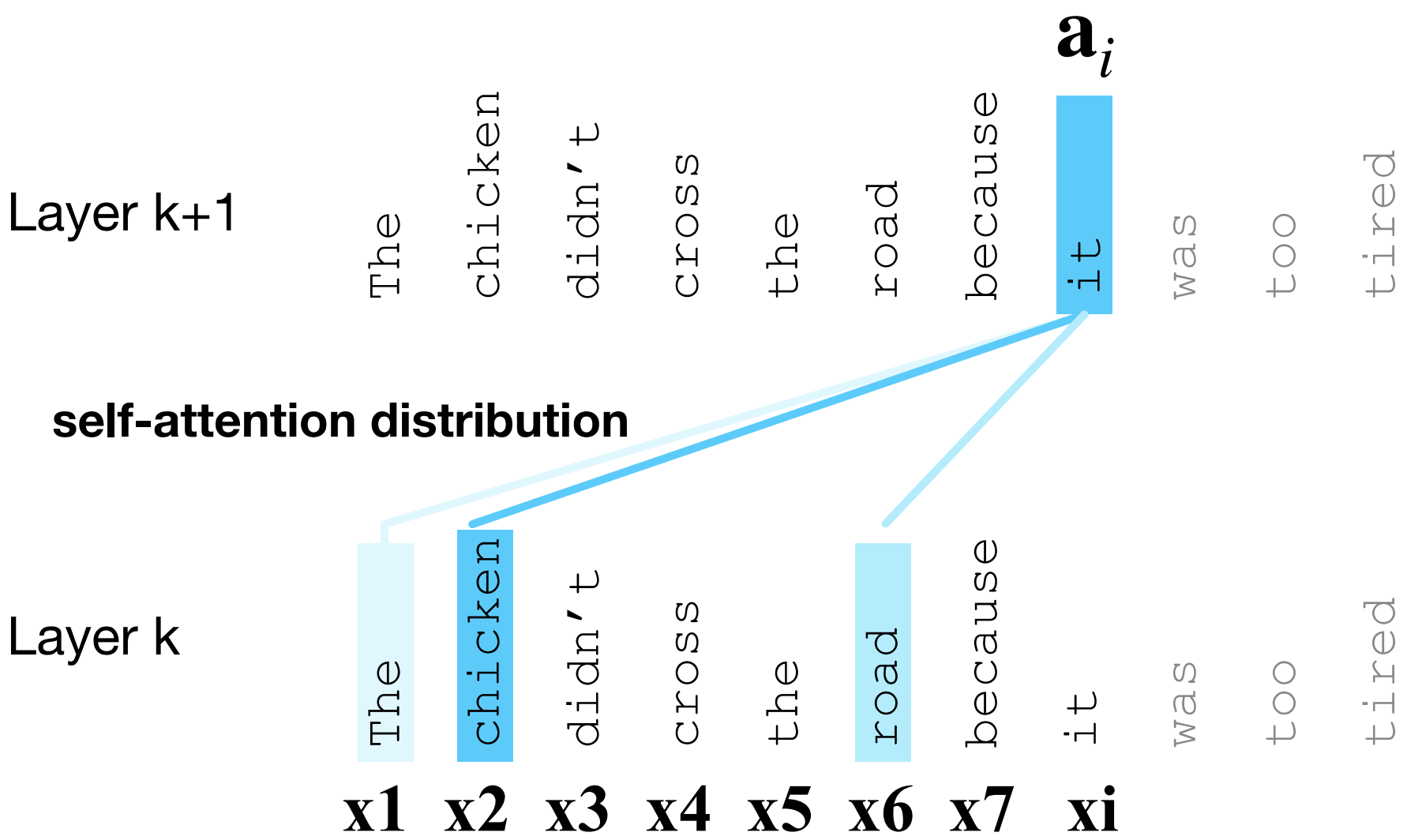
$$\mathbf{a}_i = \sum_{j \leq i} \alpha_j \cdot \mathbf{x}_j$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\alpha = \text{softmax}([\text{score}(\mathbf{x}_i, \mathbf{x}_j) \text{ for j in } 1 \ldots 7, i])$$

Intuition of attention

columns corresponding to input tokens

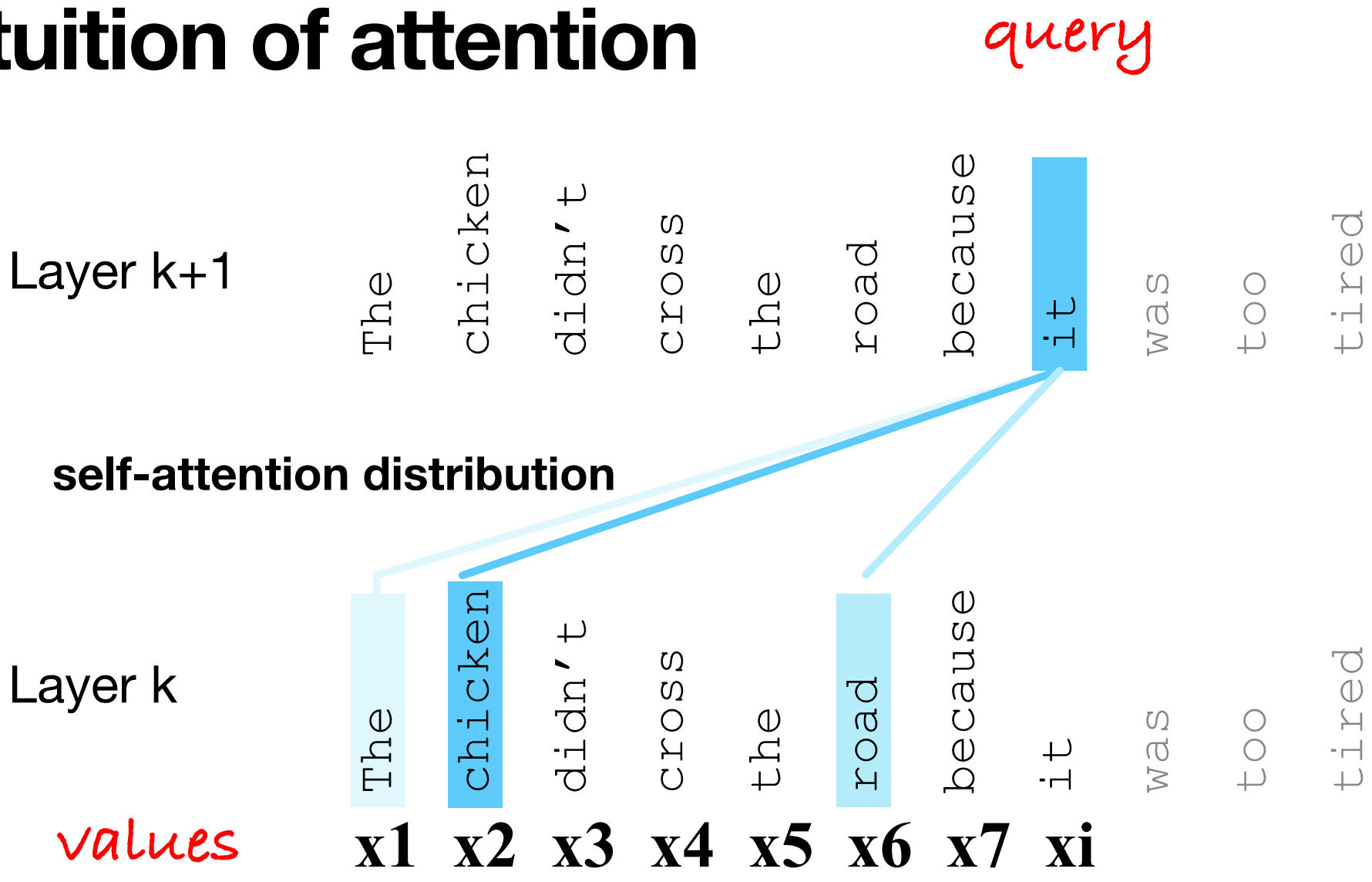


An Actual Attention Head is slightly more complicated

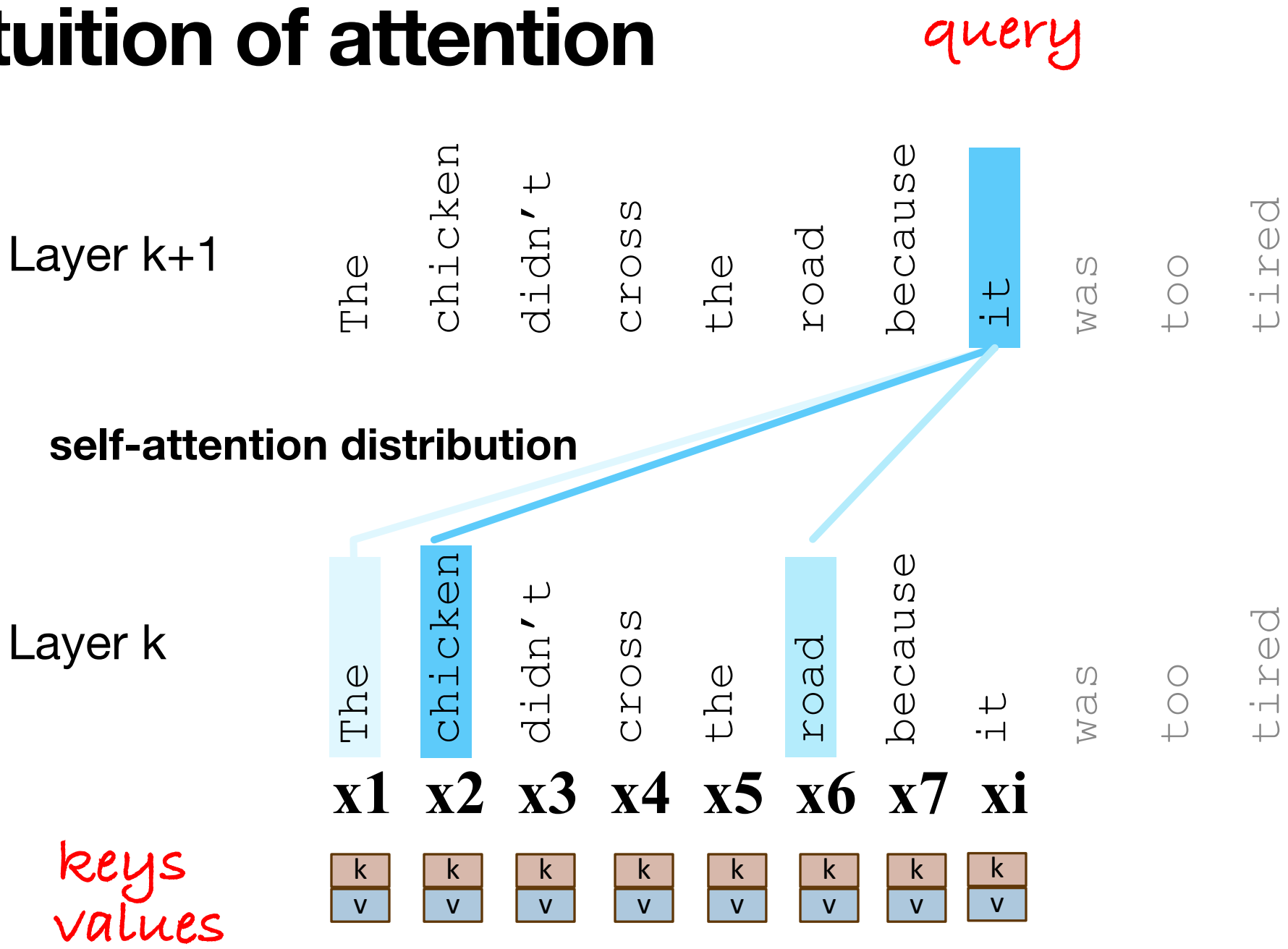
High-level idea: instead of using vectors (like x_i and x_4) directly, we'll represent 3 separate roles each vector $\mathbf{x}_i$ plays:

- **query**: As *the current element* being compared to the preceding inputs.
- **key**: as *a preceding input* that is being compared to the current element to determine a similarity
- **value**: a value of a preceding element that gets weighted and summed

Intuition of attention



Intuition of attention



Actual Attention Head

We'll use matrices to project each vector $\mathbf{x}_i$ into a representation of its role as query, key, value:

- **query**: $\mathbf{W^Q}$
- **key**: $\mathbf{W^K}$
- **value**: $\mathbf{W^V}$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \qquad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \qquad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

Actual Attention Head

Given these 3 representation of $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \qquad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \qquad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

Actual Attention Head

Given these 3 representation of $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \qquad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \qquad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

To compute similarity of current element $\mathbf{x}_i$ with some prior element $\mathbf{x}_j$

Actual Attention Head

Given these 3 representation of $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \qquad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \qquad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

To compute similarity of current element $\mathbf{x}_i$ with some prior element $\mathbf{x}_j$

We'll use dot product between $\mathbf{q}_i$ and $\mathbf{k}_j$.

Actual Attention Head

Given these 3 representation of $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \qquad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \qquad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

To compute similarity of current element $\mathbf{x}_i$ with some prior element $\mathbf{x}_j$

We'll use dot product between $\mathbf{q}_i$ and $\mathbf{k}_j$.

And instead of summing up $\mathbf{x}_j$, we'll sum up $\mathbf{v}_j$

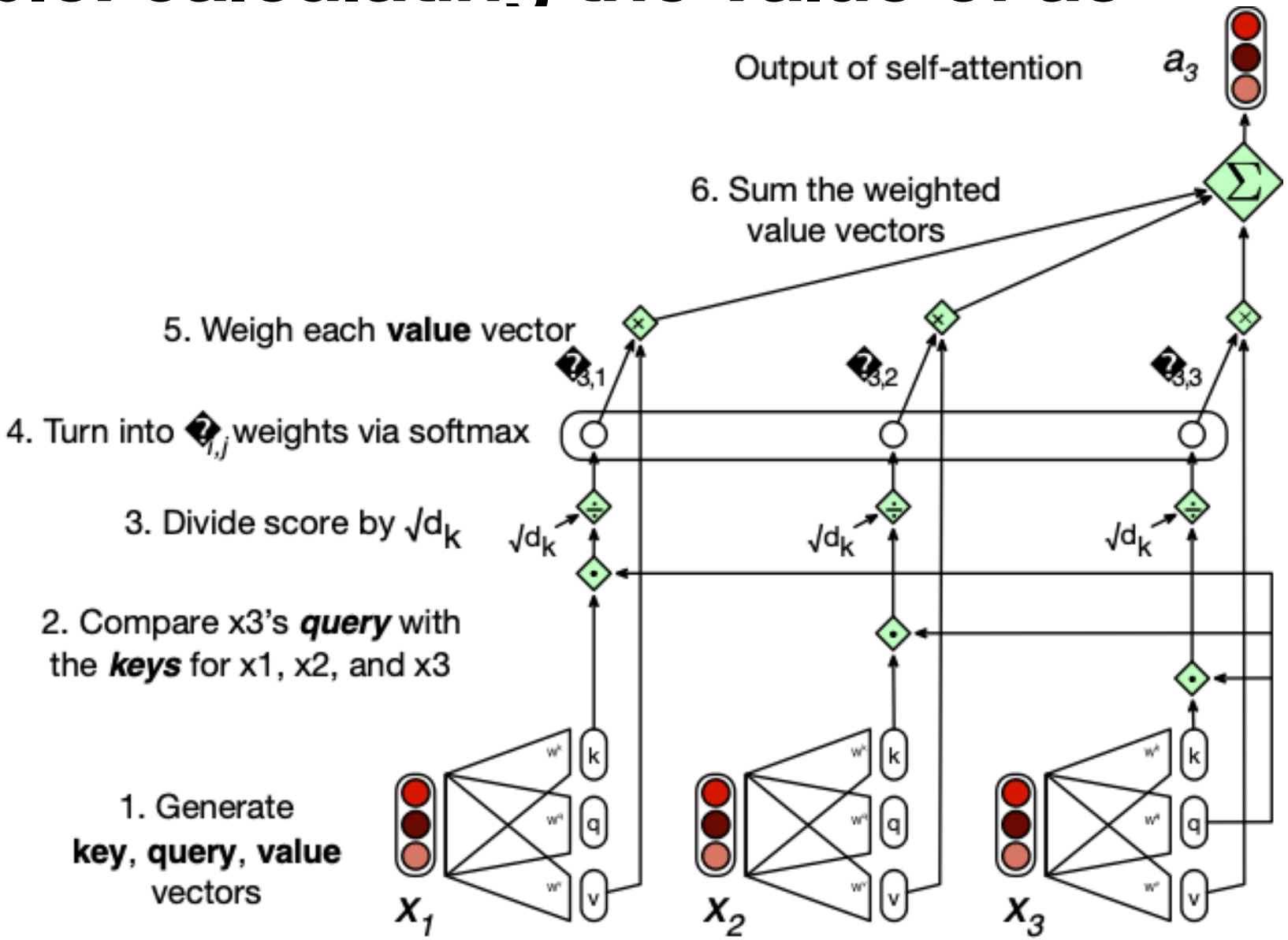
Final equations for one attention head

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \qquad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \qquad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \mathbf{k}_j^\top}{\sqrt{d^k}} \qquad \alpha = \text{softmax}([\text{score}(\mathbf{x}_i, \mathbf{x}_j) \forall\ j \leq i])$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_j \cdot \mathbf{v}_j$$

Example: calculating the value of a3



Multi-head Attention

- Instead of one attention head, we'll have lots of them!
- Intuition: each head might be attending to the context for different purposes
  - E.g., different linguistic relationships or patterns in the context

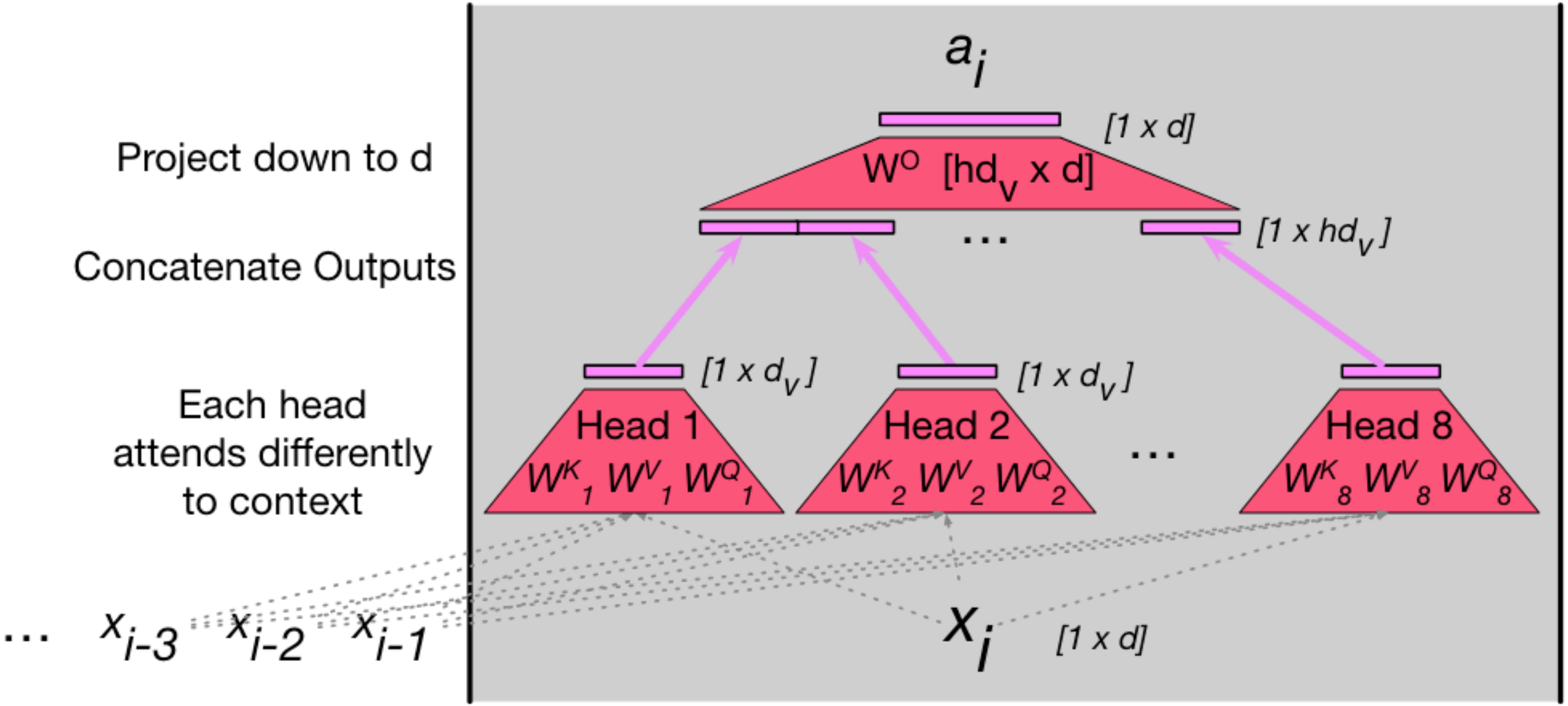
$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{Qc} \quad \mathbf{k}_i^c = \mathbf{x}_i \mathbf{W}^{Kc} \quad \mathbf{v}_i^c = \mathbf{x}_i \mathbf{W}^{Vc}$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \mathbf{k}_j^{c\top}}{\sqrt{d^k}} \qquad \alpha_i^c = \text{softmax}([\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) \forall\ j \leq i])$$

$$\textbf{head}_i^c = \sum_{j \leq i} \alpha_{i,j}^c \cdot \mathbf{v}_j^c \qquad \mathbf{a}_i = (\textbf{head}^1 \oplus \textbf{head}^2 \ldots \oplus \textbf{head}^h)\mathbf{W}^O$$

$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \ldots, \mathbf{x}_n]) = \mathbf{a}_i$$

Multi-head attention



Parallelizing computation using X

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{Q} \quad \mathbf{K} = \mathbf{X}\mathbf{W}^{K} \quad \mathbf{V} = \mathbf{X}\mathbf{W}^{V}$$

Parallelizing computation using X

For attention/transformer block we've been computing a **single** output at a **single** time step *i* in a **single** residual stream.

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{Q} \quad \mathbf{K} = \mathbf{X}\mathbf{W}^{K} \quad \mathbf{V} = \mathbf{X}\mathbf{W}^{V}$$

Parallelizing computation using X

For attention/transformer block we've been computing a **single** output at a **single** time step *i* in a **single** residual stream.

But we can pack the *N* tokens of the input sequence into a single matrix **X** of size [$N \times d$].

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q \qquad \mathbf{K} = \mathbf{X}\mathbf{W}^K \qquad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

Parallelizing computation using X

For attention/transformer block we've been computing a **single** output at a **single** time step *i* in a **single** residual stream.

But we can pack the *N* tokens of the input sequence into a single matrix **X** of size [$N \times d$].

Each row of X is the embedding of one token of the input.

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

Parallelizing computation using X

For attention/transformer block we've been computing a **single** output at a **single** time step *i* in a **single** residual stream.

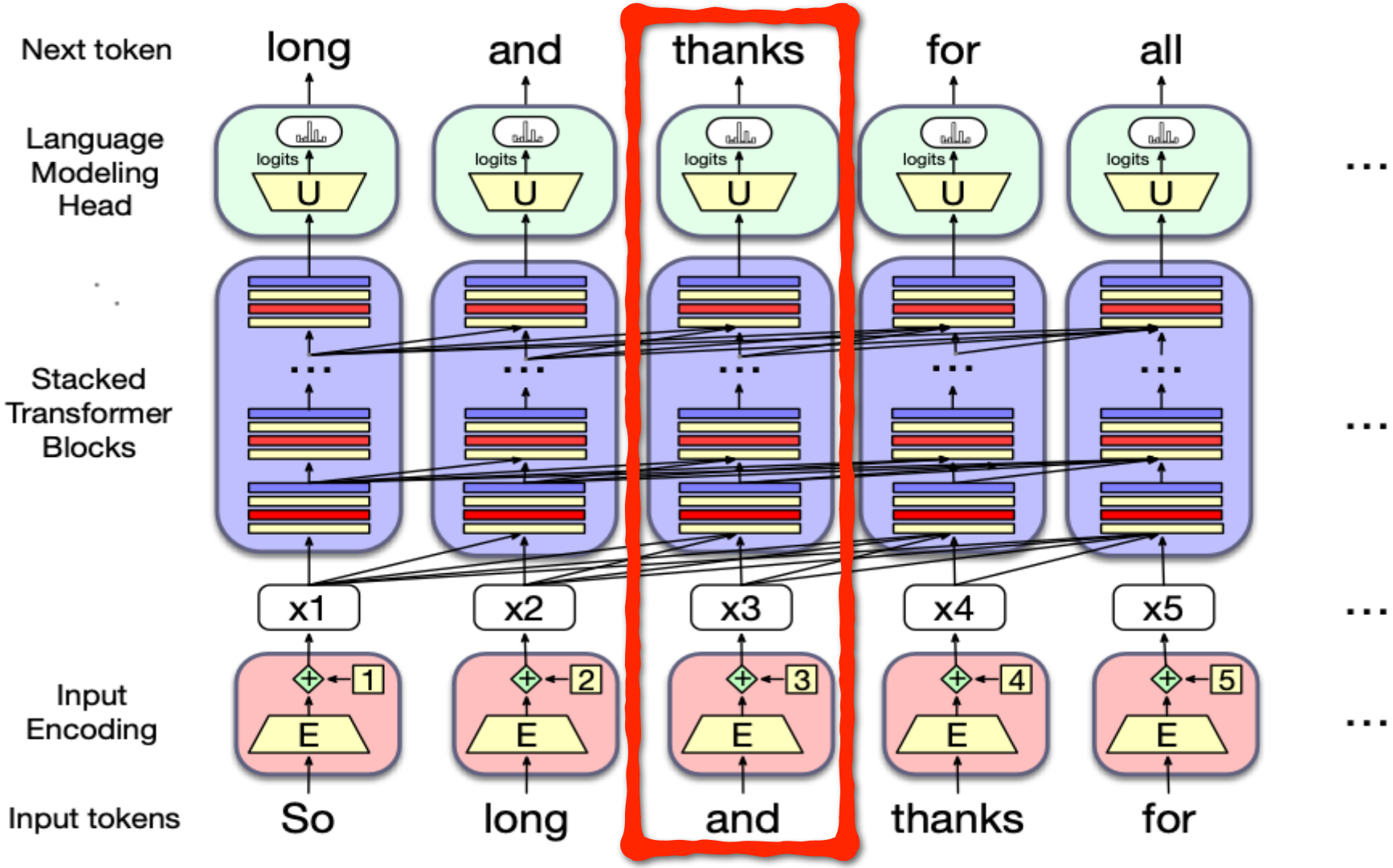
But we can pack the *N* tokens of the input sequence into a single matrix **X** of size [$N \times d$].

Each row of X is the embedding of one token of the input.

X can have 1K-32K rows, each of the dimensionality of the embedding *d* (the **model dimension**)

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q \qquad \mathbf{K} = \mathbf{X}\mathbf{W}^K \qquad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

Let's consider what happens to the embeddings for one word



QK^T

Now can do a single matrix multiply to combine Q and K^T

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \mathbf{k}_j^\top}{\sqrt{d^k}}$$

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d^k}}$$

N

q1•k1	q1•k2	q1•k3	q1•k4
q2•k1	q2•k2	q2•k3	q2•k4
q3•k1	q3•k2	q3•k3	q3•k4
q4•k1	q4•k2	q4•k3	q4•k4

N

Parallelizing attention

- Use **Q** and **K** matrices to calculate scores, take the softmax, and then multiply the result by **V** resulting in matrix of shape $N \times d$
  - An attention vector for each input token

$$\mathbf{A} = \text{softmax}\left(\mathbb{M}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d^k}}\right)\right)\mathbf{V}$$

Masking out the future

- What is this mask function?
  QK$^{\mathbf{T}}$ has a score for each query-key pair, *including those that follow the query*.
- Guessing the next word is pretty simple if you already know it!

$$\mathbf{A} = \text{softmax}\left(\mathbb{M}\left(\frac{\mathbf{QK}^\top}{\sqrt{d^k}}\right)\right)\mathbf{V}$$

Masking out the future

Add –∞ to cells in upper triangle

The log softmax will turn it to 0

$$\mathbf{A} = \text{softmax}\left(\mathbb{M}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d^k}}\right)\right)\mathbf{V}$$

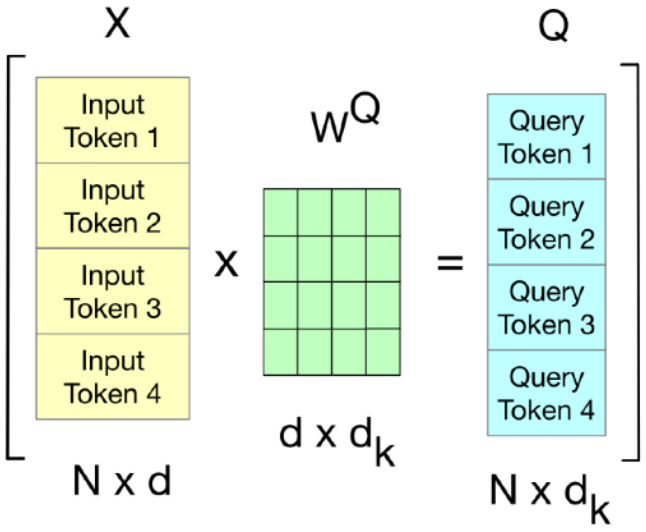
N

q1•k1	−∞	−∞	−∞
q2•k1	q2•k2	−∞	−∞
q3•k1	q3•k2	q3•k3	−∞
q4•k1	q4•k2	q4•k3	q4•k4

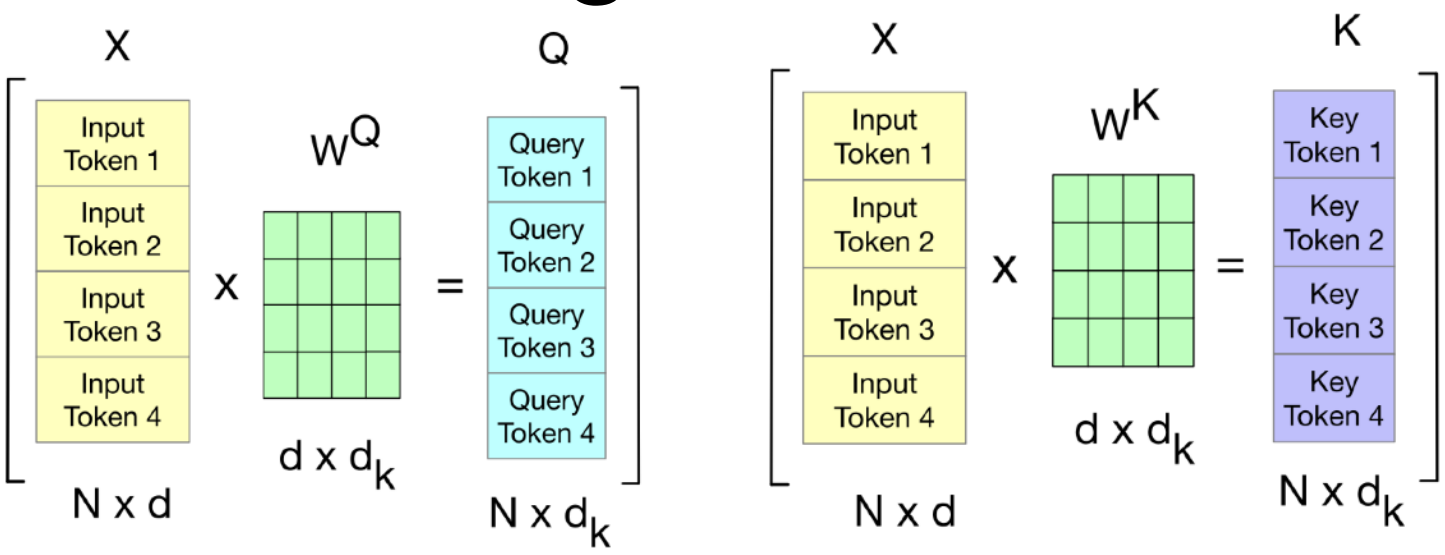
N

Attention again

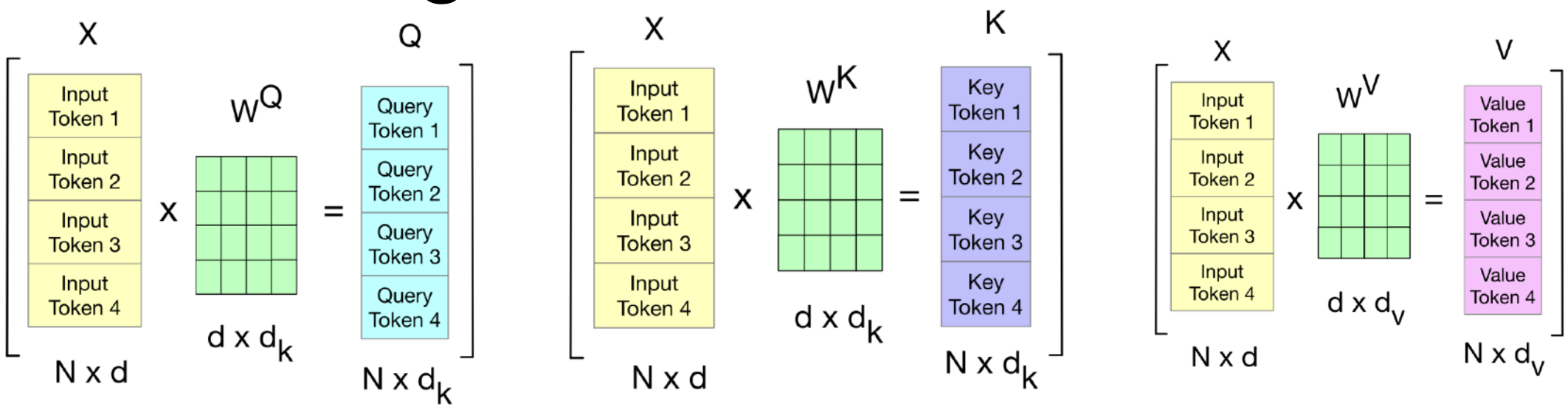
Attention again



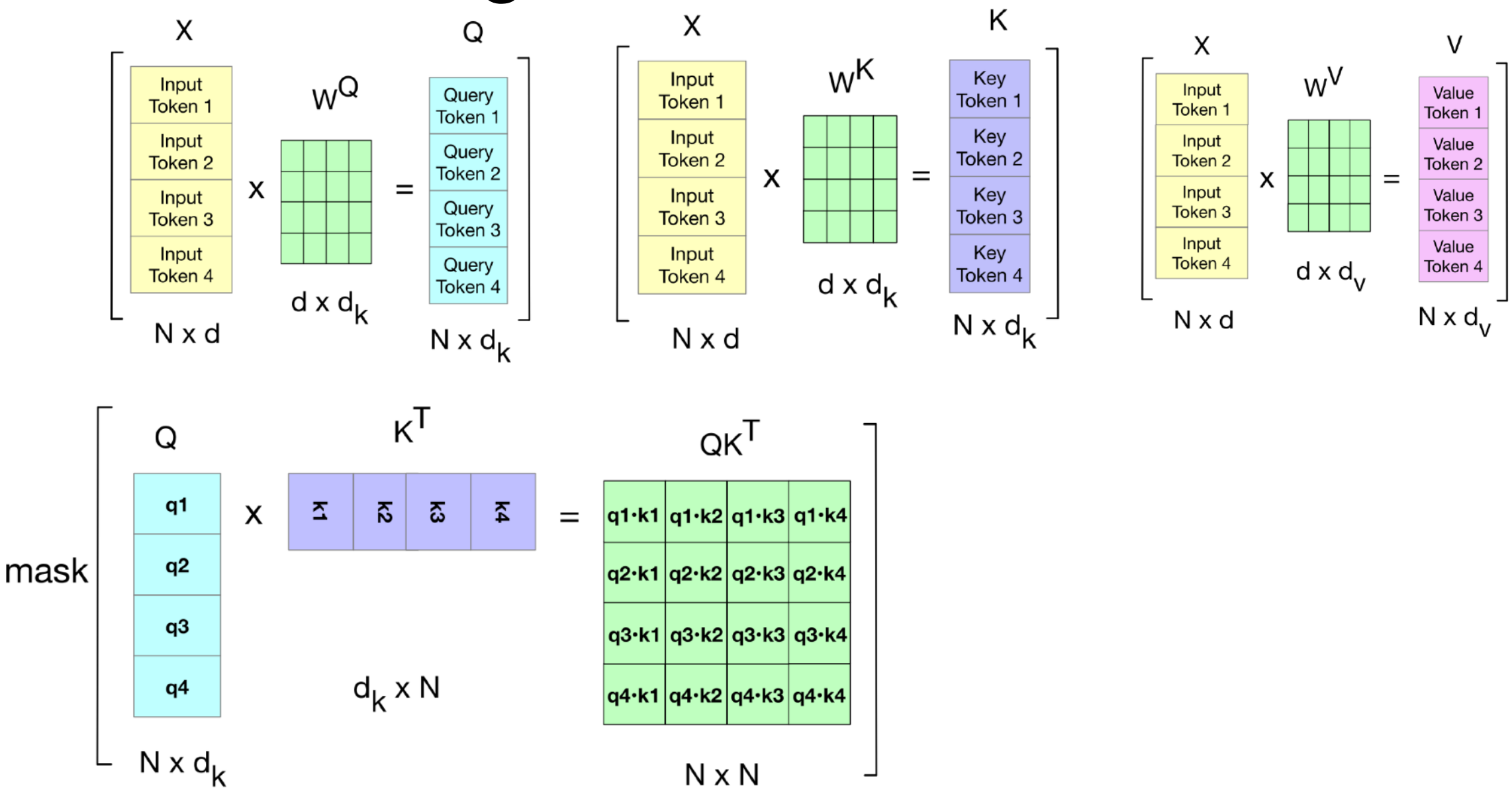
Attention again



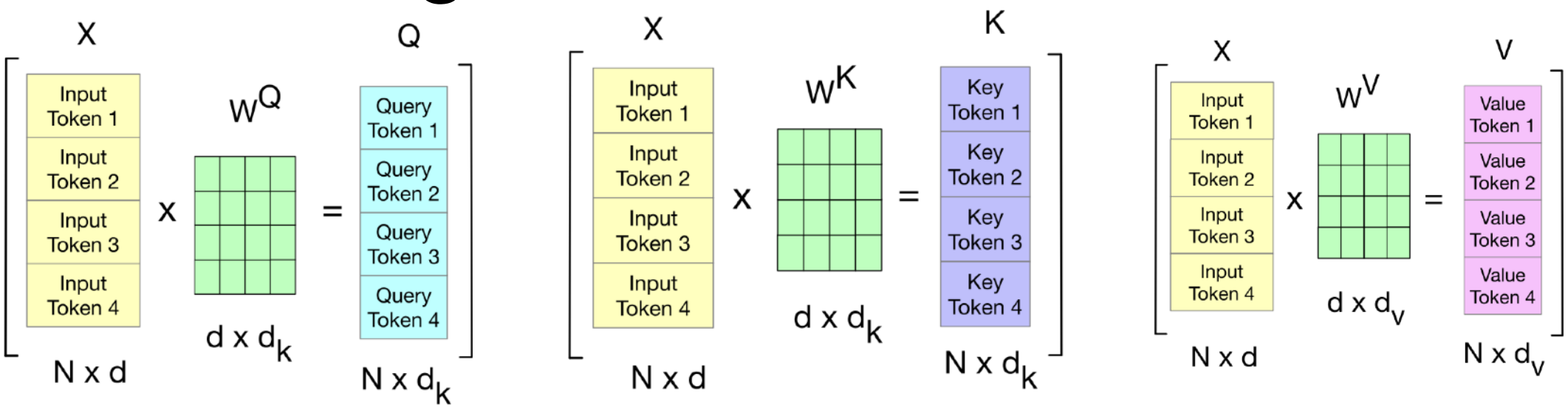
Attention again



Attention again



Attention again

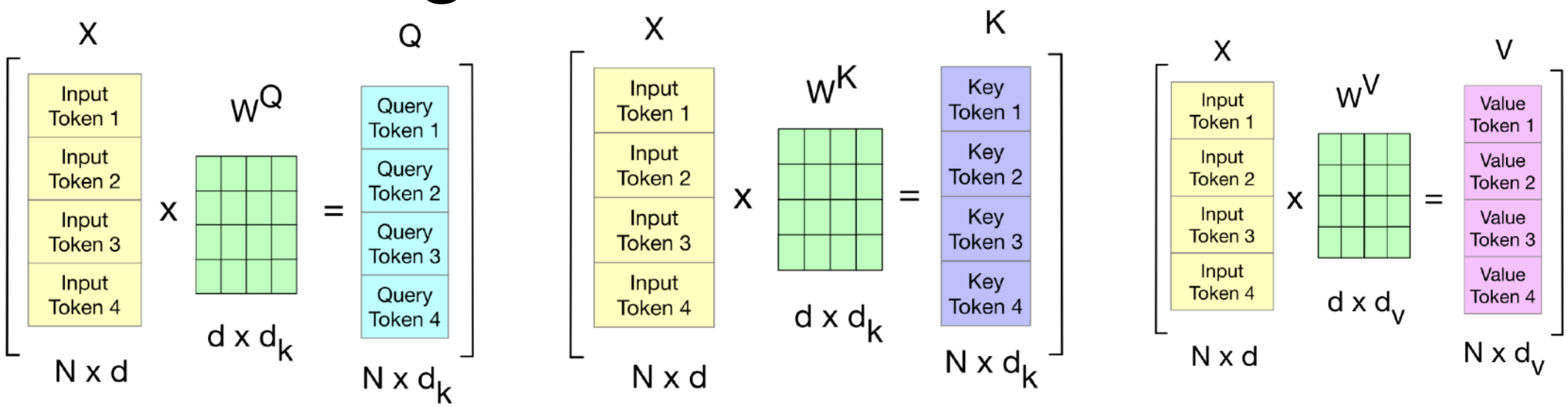


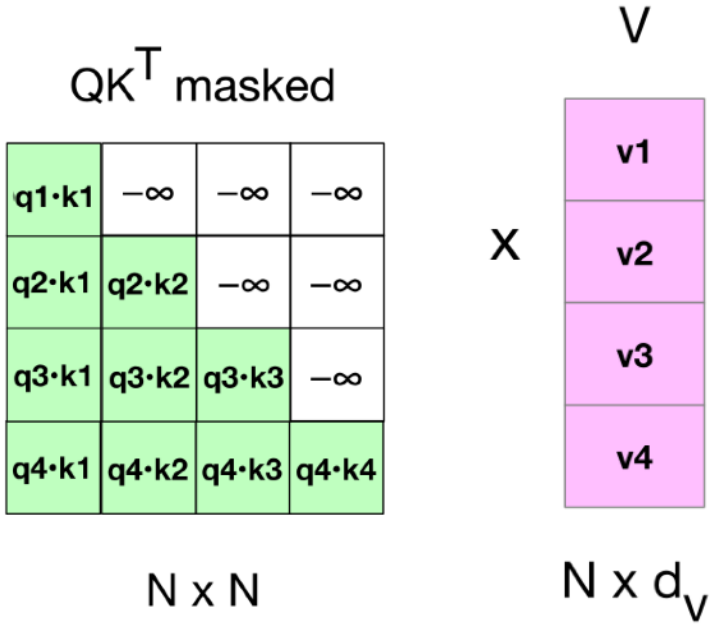
QK^T masked

q1·k1	−∞	−∞	−∞
q2·k1	q2·k2	−∞	−∞
q3·k1	q3·k2	q3·k3	−∞
q4·k1	q4·k2	q4·k3	q4·k4

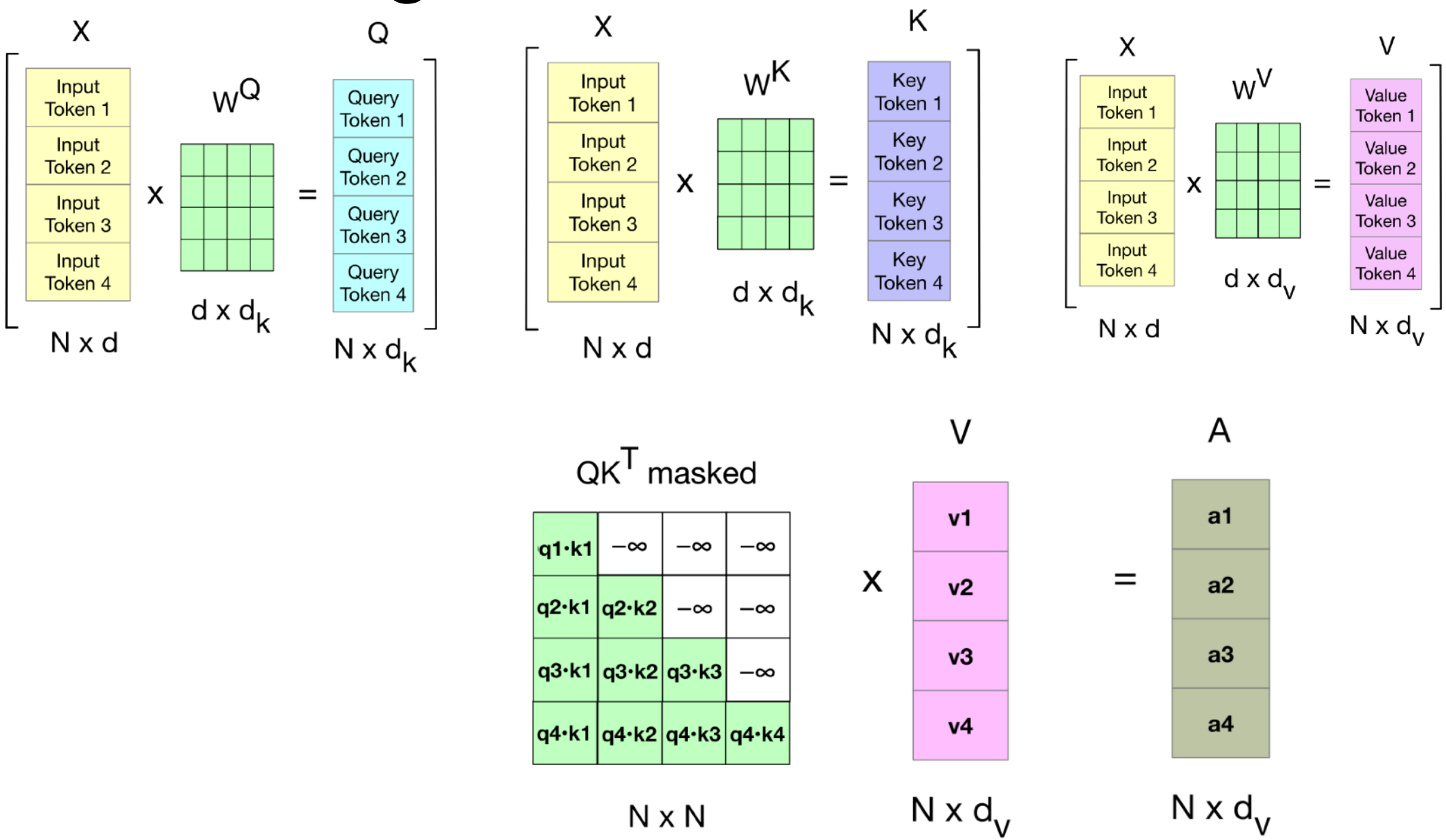
N x N

Attention again





Attention again



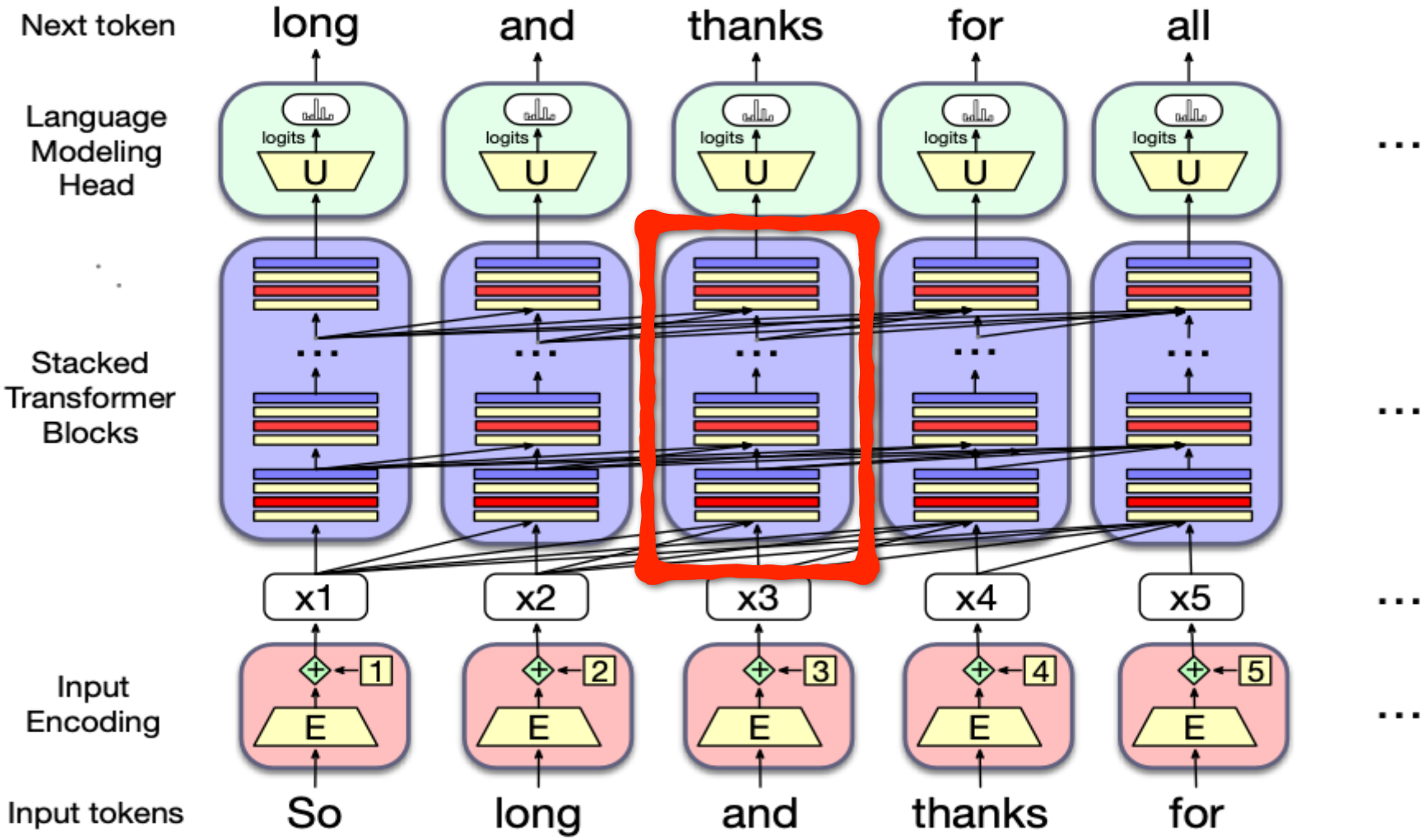
Parallelizing Multi-head Attention

$$\mathbf{Q^i} = \mathbf{XW^{Qi}}\,; \quad \mathbf{K^i} = \mathbf{XW^{Ki}}\,; \quad \mathbf{V^i} = \mathbf{XW^{Vi}}$$

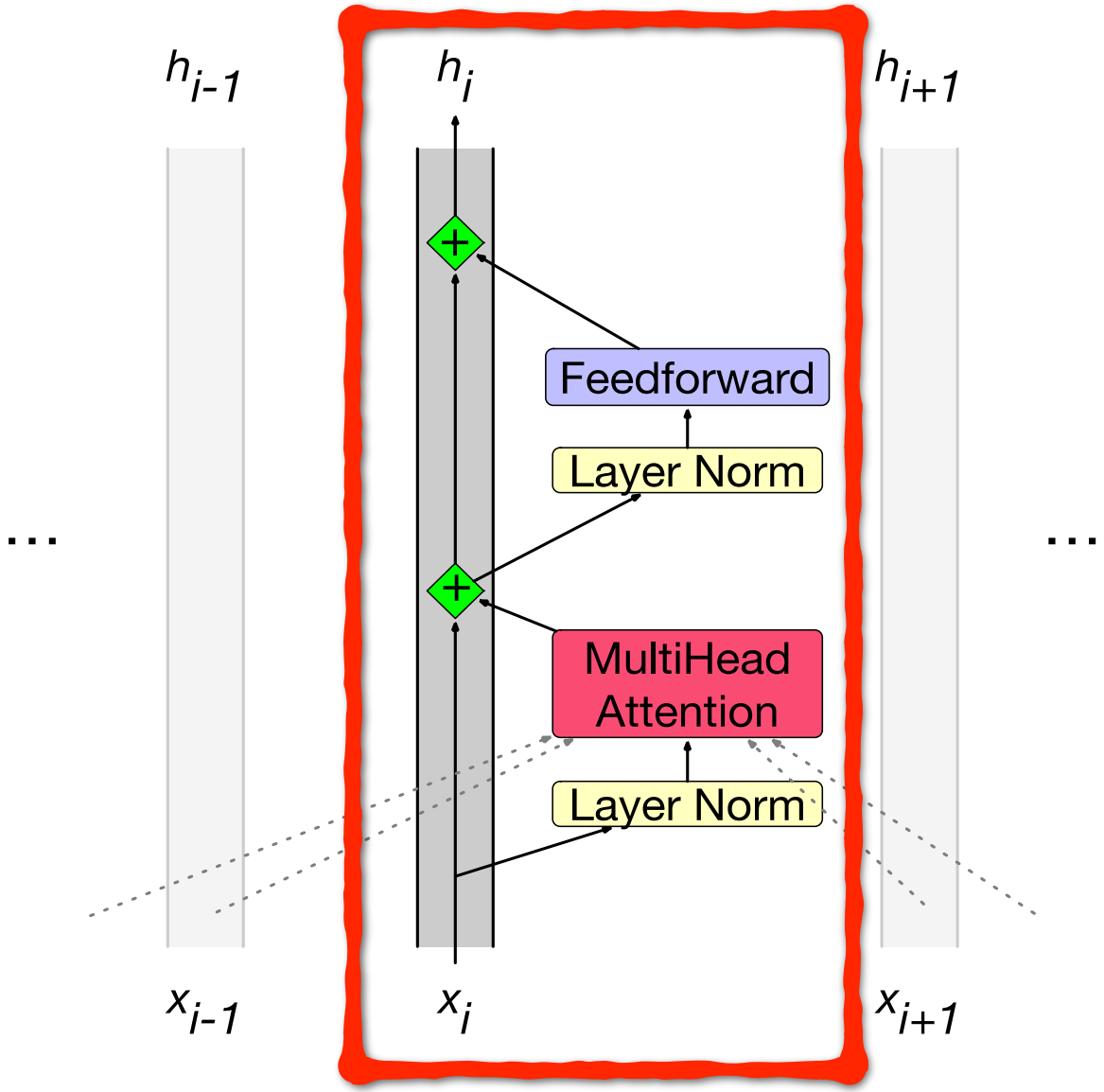
$$\mathbf{head}_i = \mathrm{SelfAttention}(\mathbf{Q^i}, \mathbf{K^i}, \mathbf{V^i}) = \mathrm{softmax}\left(\frac{\mathbf{Q^i}\mathbf{K^i}^{\mathsf{T}}}{\sqrt{d_k}}\right)\mathbf{V^i}$$

$$\mathrm{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \ldots \oplus \mathbf{head}_A)\mathbf{W^O}$$

Reminder: transformer architecture

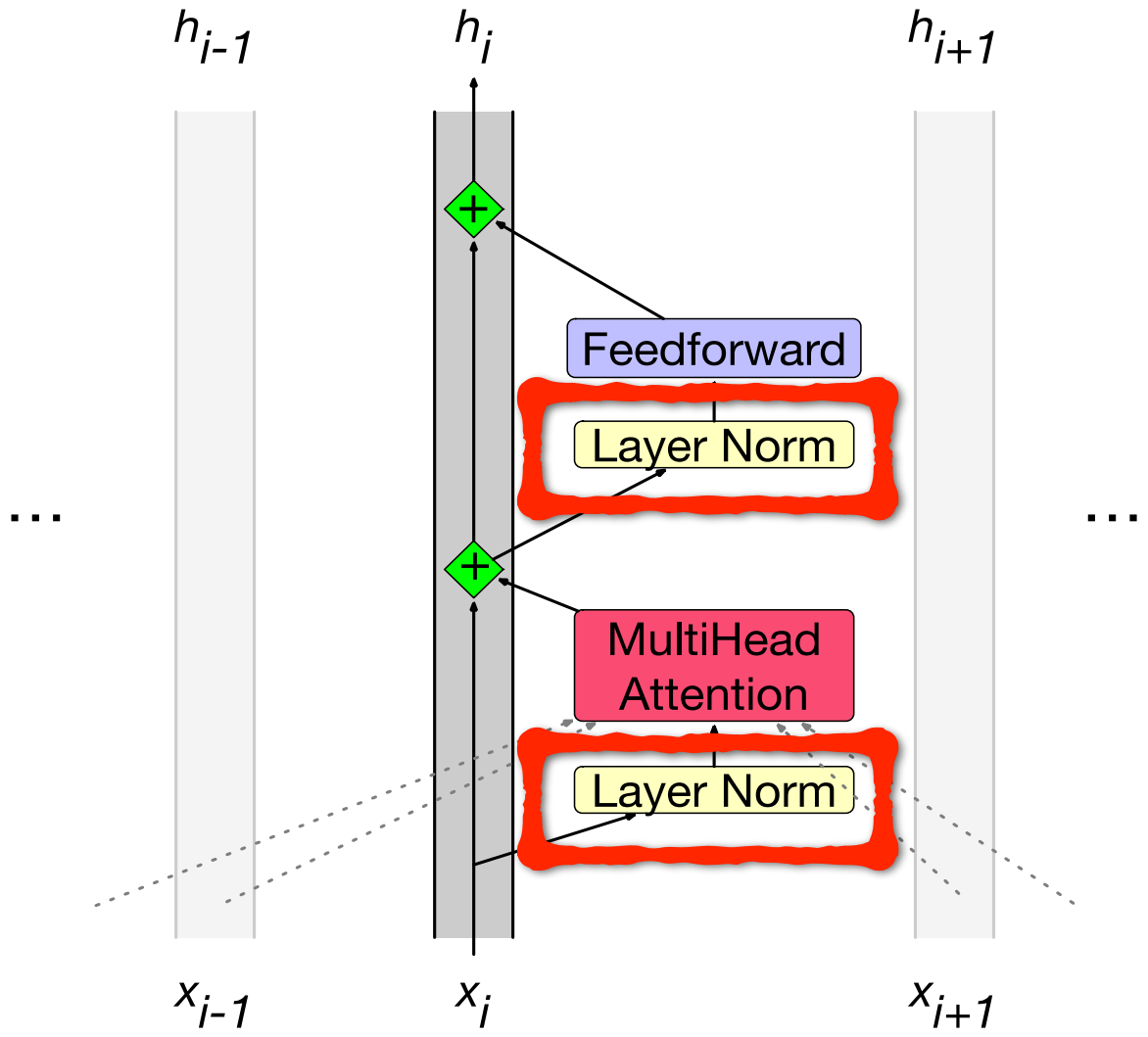


A single transformer block



Sublayers of the transformer block: Layer Norm

$$\text{LayerNorm}(\mathbf{x}_i) = \ldots$$



Layer Norm

Layer norm is a variation of the z-score from statistics, applied to a single vector in a hidden layer

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

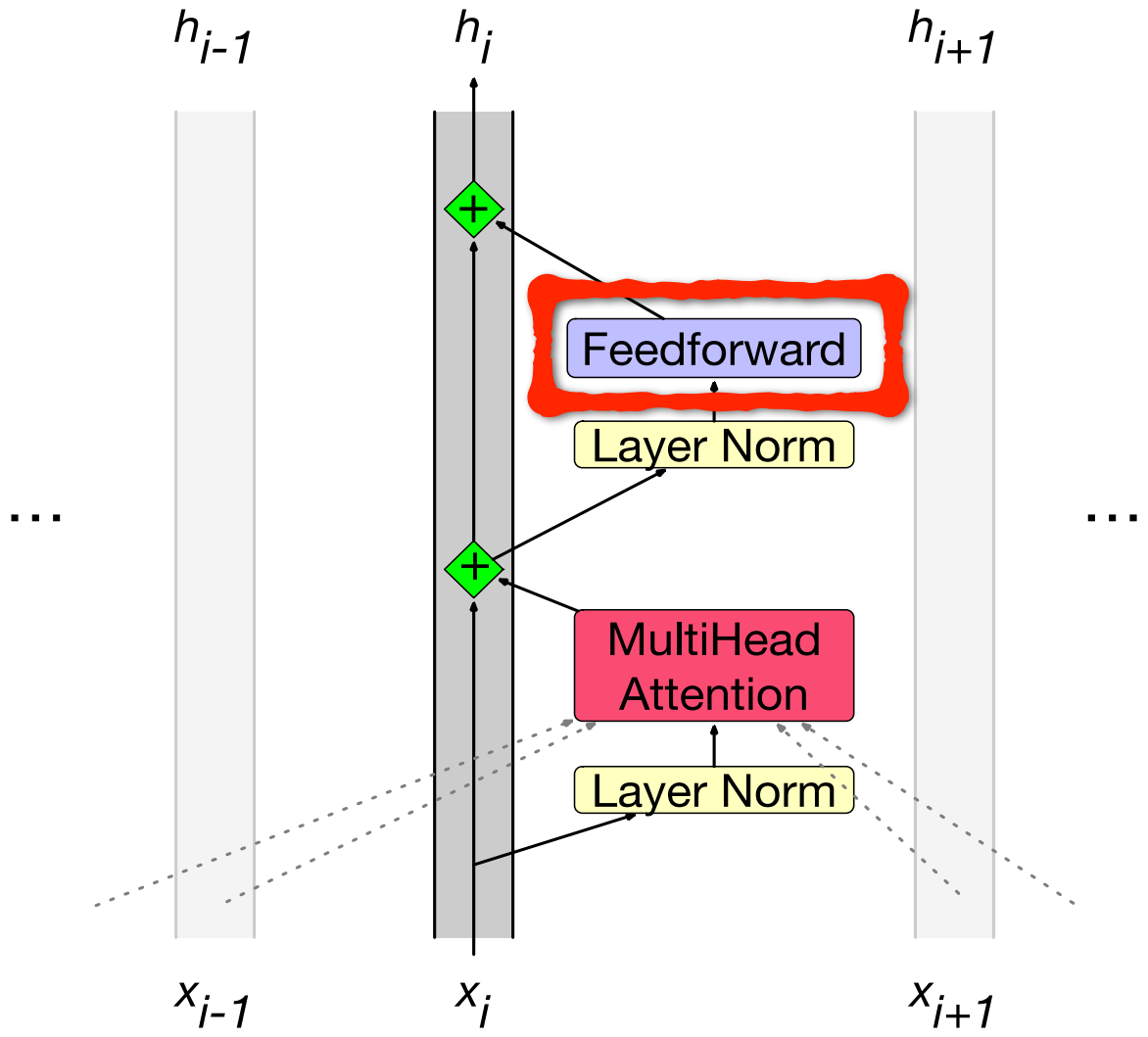
$$\sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

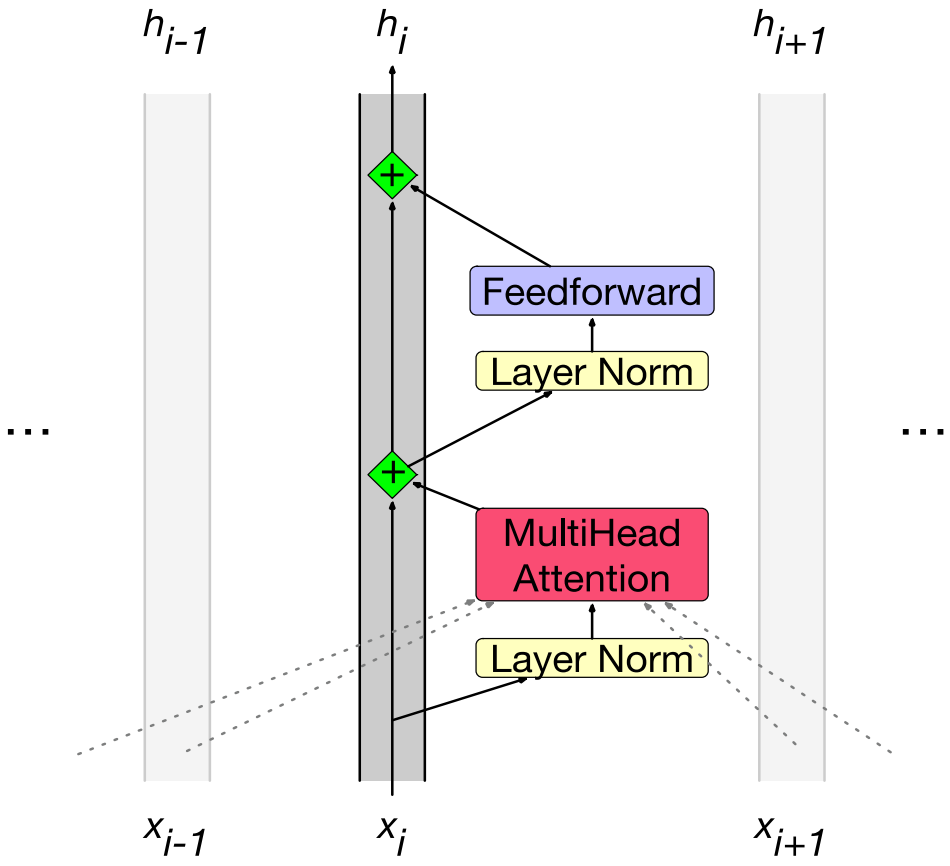
$$\text{LayerNorm}(\mathbf{x}) = \gamma\,\frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$

Sublayers of the transformer block: FFN

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$



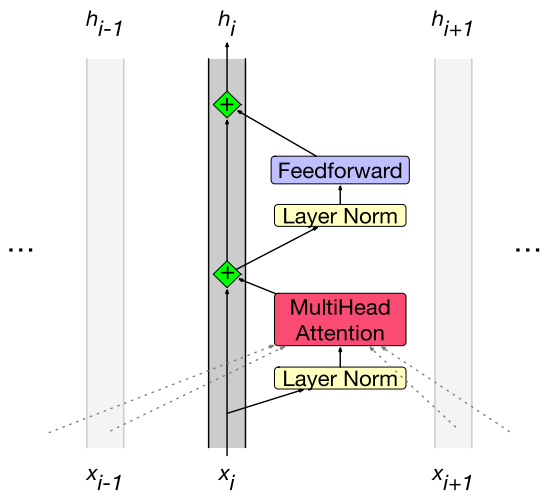
Putting together a single transformer block



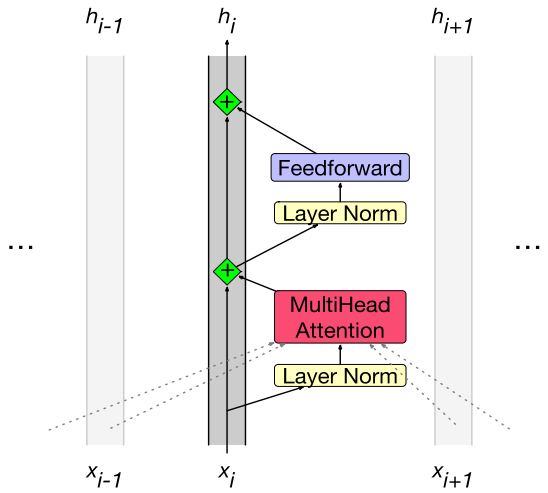
$$
\begin{aligned}
\mathbf{t}_i^1 &= \mathrm{LayerNorm}(\mathbf{x}_i) \\
\mathbf{t}_i^2 &= \mathrm{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{t}_1^1, \cdots, \mathbf{t}_N^1]) \\
\mathbf{t}_i^3 &= \mathbf{t}_i^2 + \mathbf{x}_i \\
\mathbf{t}_i^4 &= \mathrm{LayerNorm}(\mathbf{t}_i^3) \\
\mathbf{t}_i^5 &= \mathrm{FFN}(\mathbf{t}_i^4) \\
\mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3
\end{aligned}
$$

A transformer is a stack of these blocks so all the vectors are of the same dimensionality d

Block 2



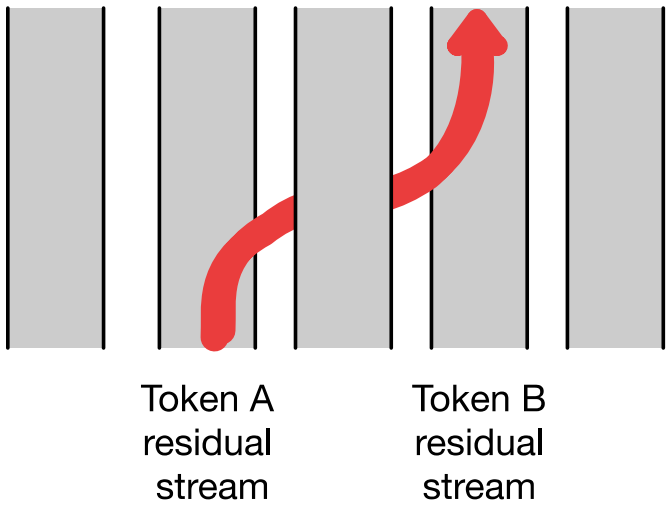
Block 1



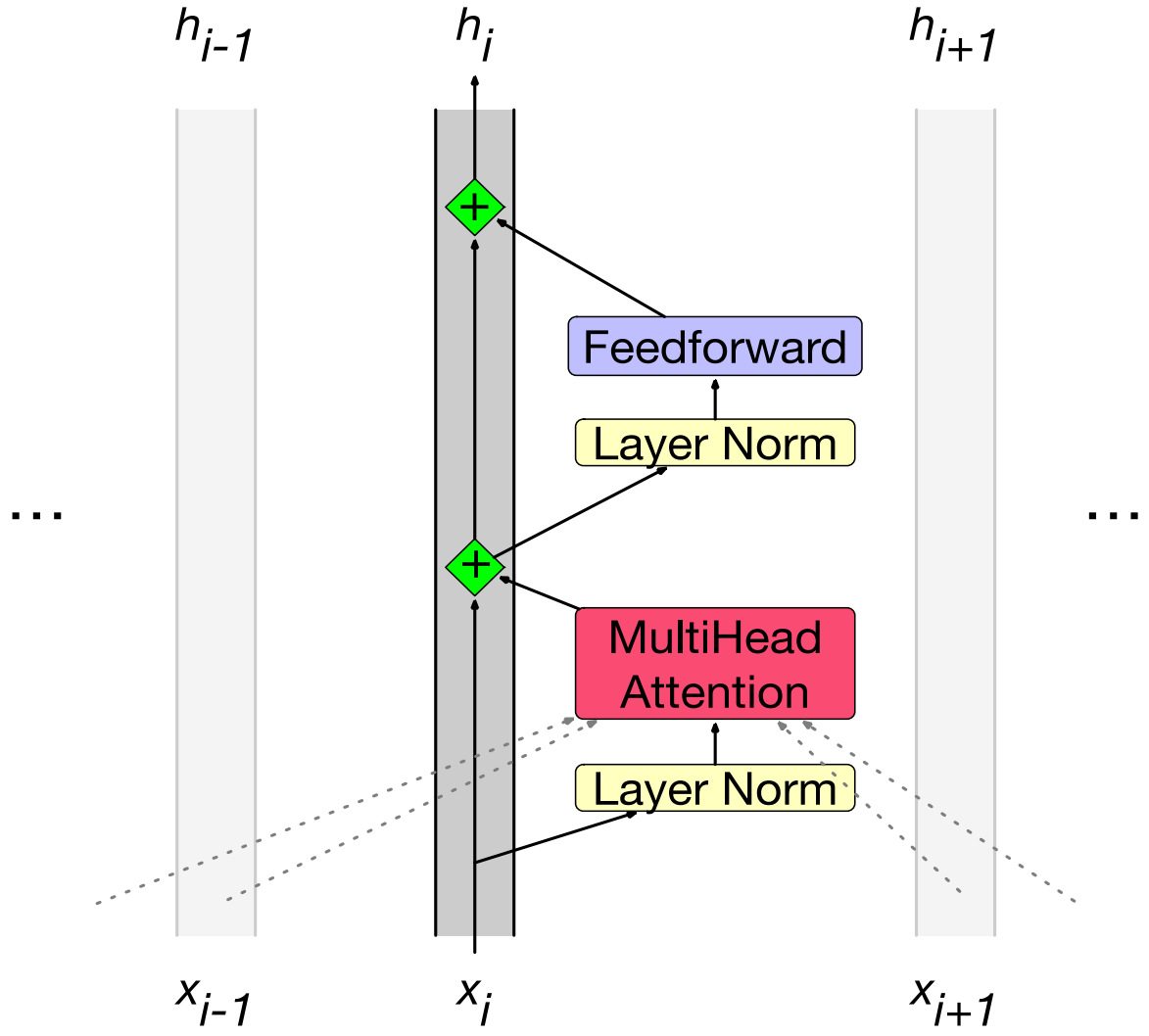
Residual streams and attention

Notice that all parts of the transformer block apply to 1 residual stream except attention, which takes information from other tokens

Elhage et al. (2021) show that we can view attention heads as literally moving information from the residual stream of a neighboring token into the current stream



Residual stream view



- FFN and attention layers **read** from and **write** to the residual stream
- FFN layers have access to “one lane” only. Same computation applied on every “lane”
- Attention layers can read from from other “lanes” too
- $\mathbf{x}_i$ is transformed into $\mathbf{h}_i^L$ through a sequence of non-linear transformations

Putting together a single transformer block

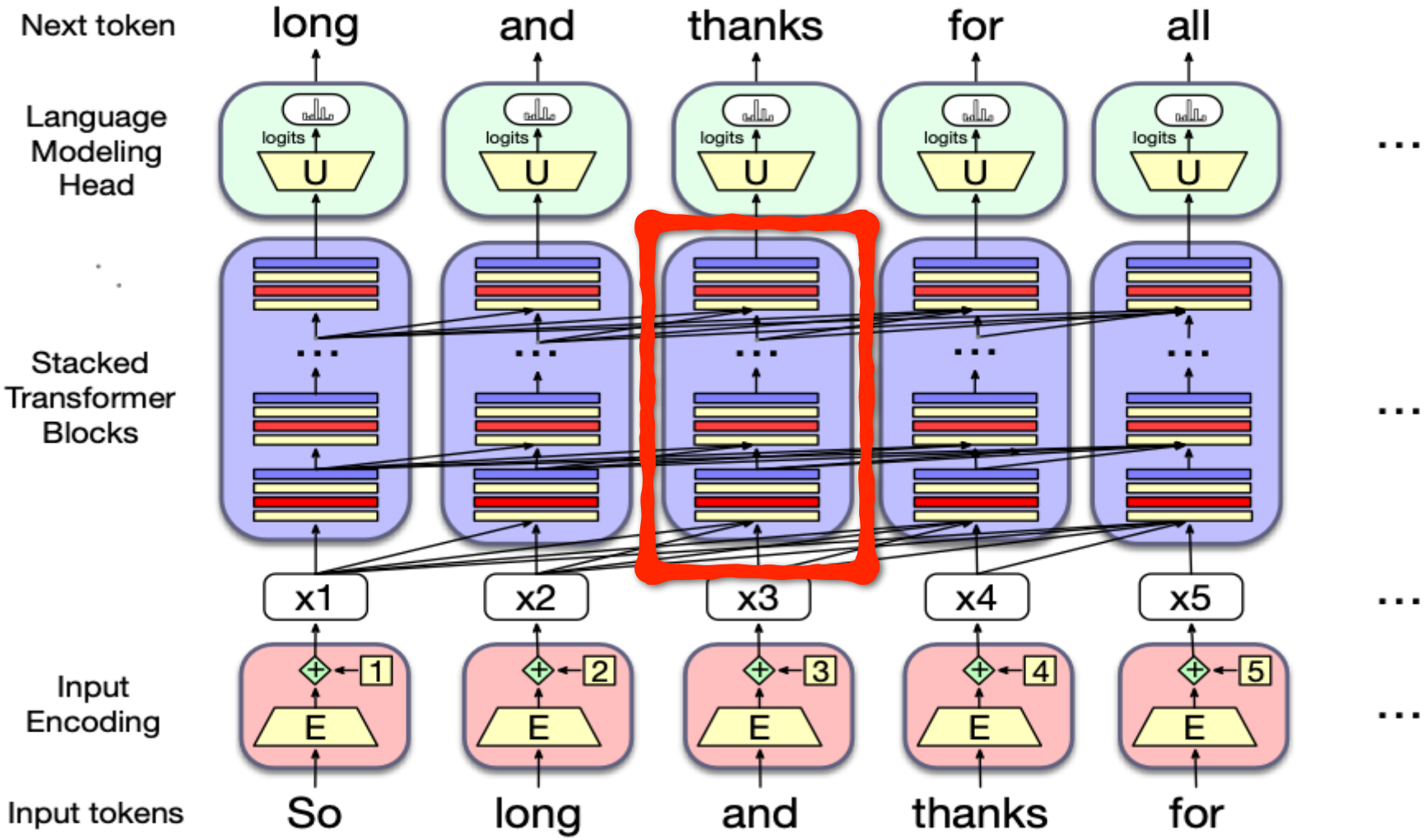
Single vector:

$$
\begin{aligned}
\mathbf{t}_i^1 &= \mathrm{LayerNorm}(\mathbf{x}_i) \\
\mathbf{t}_i^2 &= \mathrm{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{t}_1^1, \cdots, \mathbf{t}_N^1]) \\
\mathbf{t}_i^3 &= \mathbf{t}_i^2 + \mathbf{x}_i \\
\mathbf{t}_i^4 &= \mathrm{LayerNorm}(\mathbf{t}_i^3) \\
\mathbf{t}_i^5 &= \mathrm{FFN}(\mathbf{t}_i^4) \\
\mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3
\end{aligned}
$$

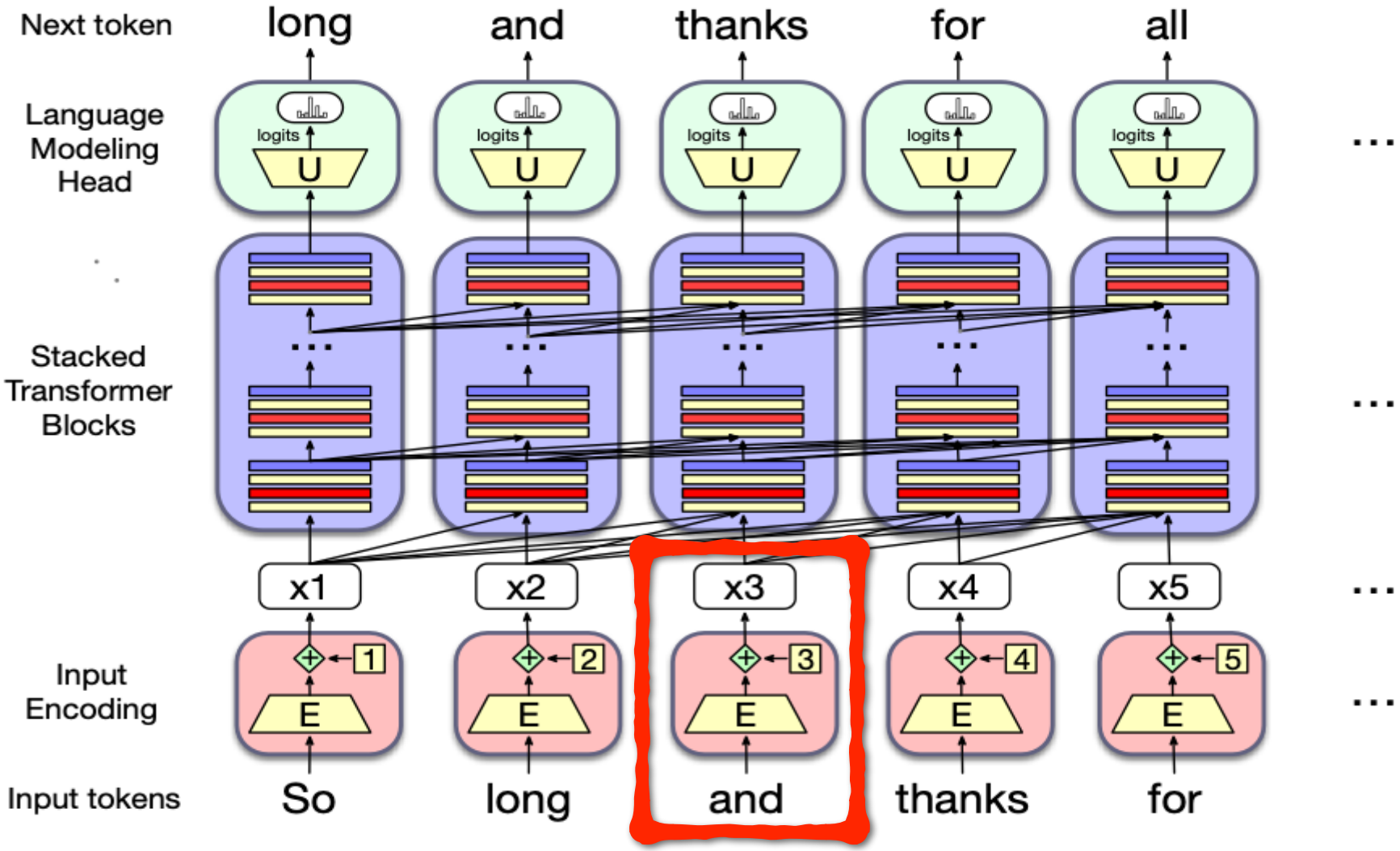
Matrix of inputs:

$$
\begin{aligned}
\mathbf{T}^1 &= \mathrm{LayerNorm}(\mathbf{X}) \\
\mathbf{T}^2 &= \mathrm{MultiHeadAttention}(\mathbf{T}^1) \\
\mathbf{T}^3 &= \mathbf{T}^2 + \mathbf{X} \\
\mathbf{T}^4 &= \mathrm{LayerNorm}(\mathbf{T}^3) \\
\mathbf{T}^5 &= \mathrm{FFN}(\mathbf{T}^4) \\
\mathbf{H} &= \mathbf{T}^5 + \mathbf{T}^3
\end{aligned}
$$

Reminder: transformer architecture



Reminder: transformer architecture

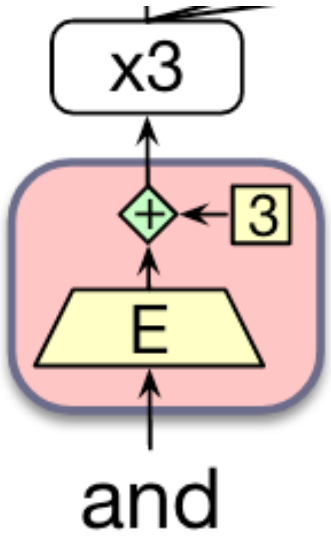


Token and Position Embeddings

The matrix $\mathbf{X}$ (of shape $N \times d$) has an embedding for each word in the context.

This embedding is created by adding two distinct embedding for each input

- token embedding
- positional embedding



Token Embeddings

Embedding matrix $\mathbf{E}$ has shape $|\mathscr{V}| \times d$.

- One row for each of the $|\mathscr{V}|$ tokens in the vocabulary.
- Each word is a row vector of *d* dimensions

Given: string "*Thanks for all the*"

1. Tokenize with BPE and convert into vocab indices

input_ids = [5,4000,10532,2224]

2. Select the corresponding rows from $\mathbf{E}$, each row an embedding

- (row 5, row 4000, row 10532, row 2224).

Position Embeddings

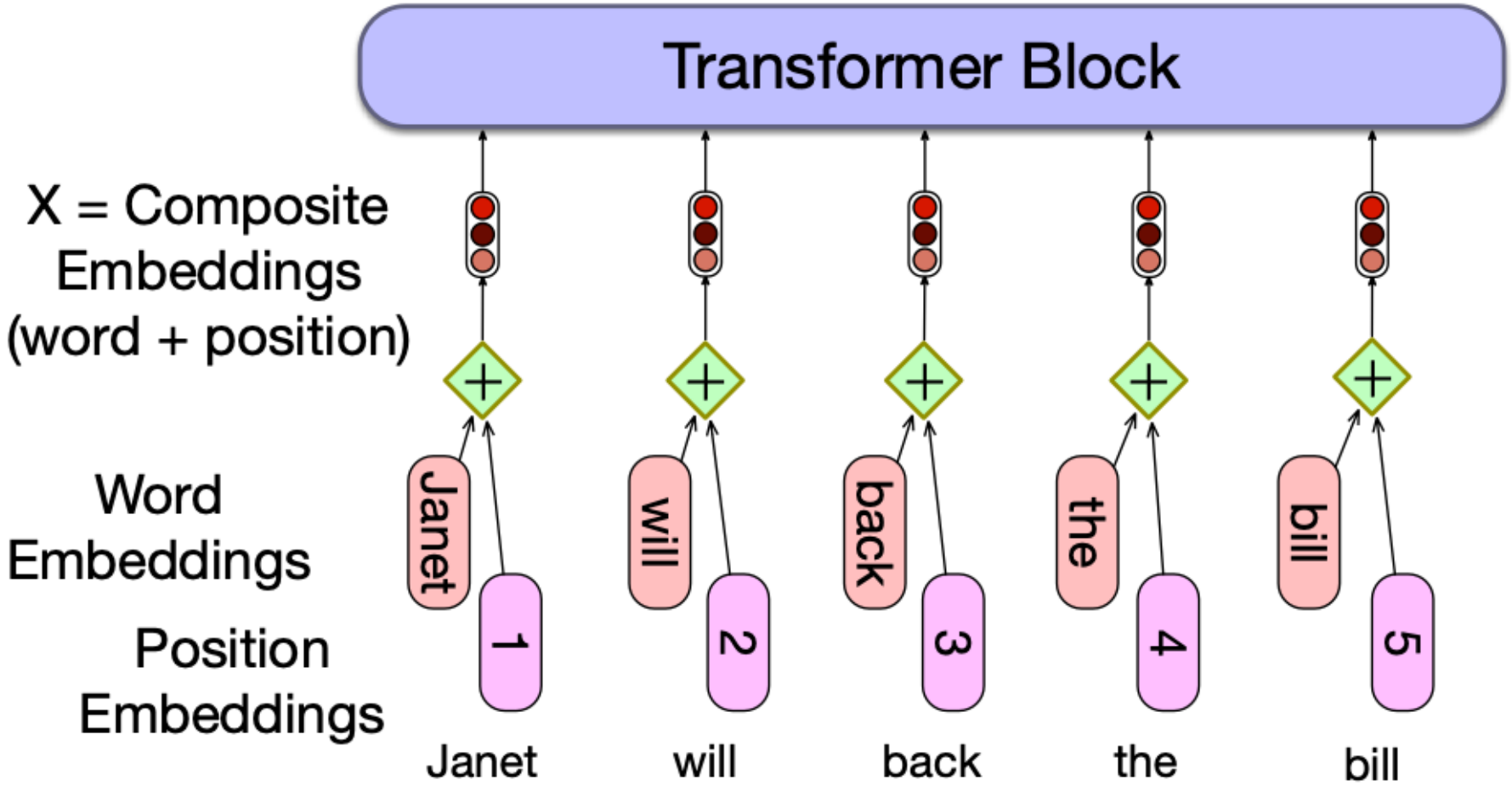
There are many methods, but we'll just describe the simplest: absolute position.

Goal: learn a position embedding matrix $\mathbf{E}_{\text{pos}}$ of shape $1 \times N$.

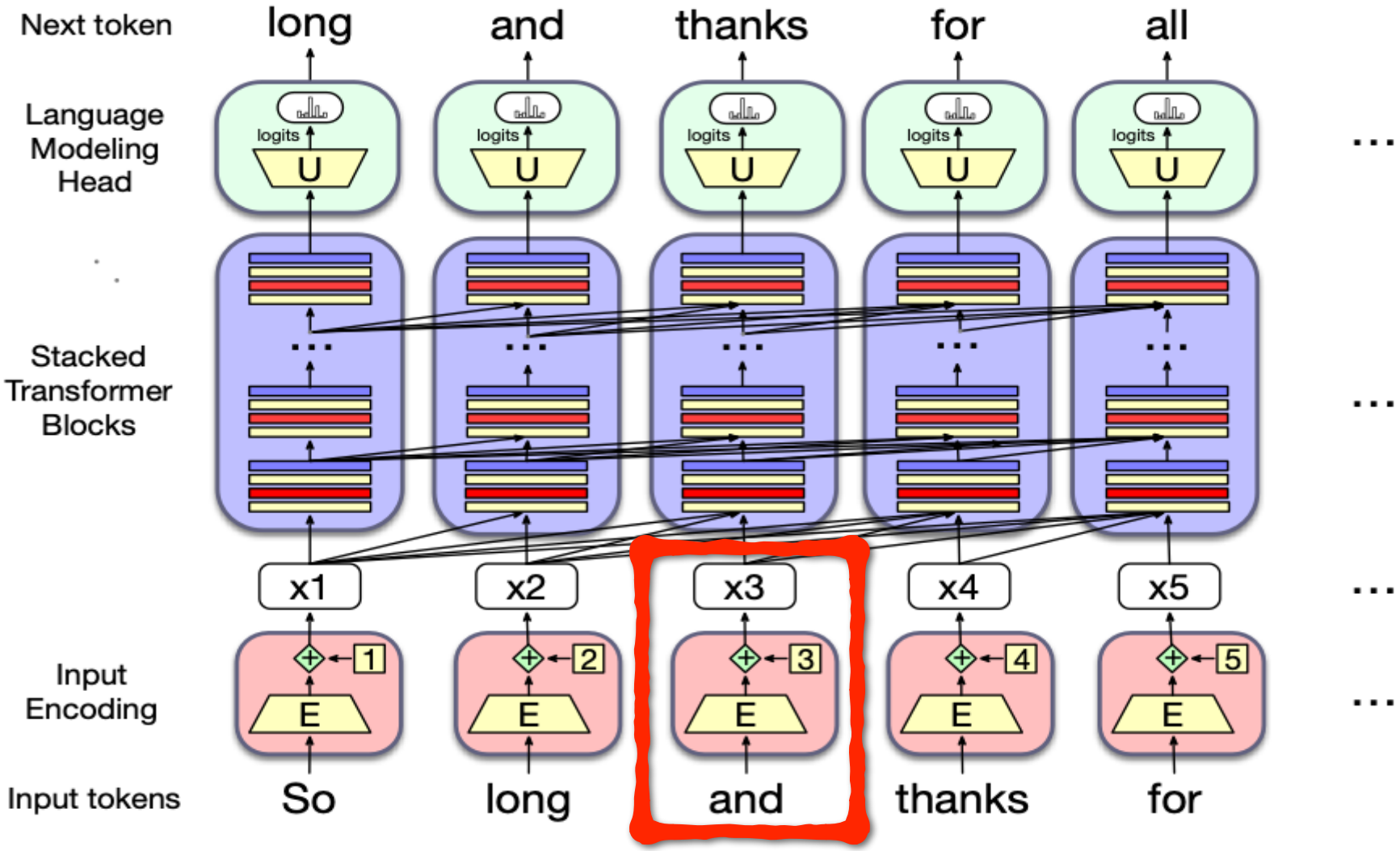
Start with randomly initialized embeddings

- one for each integer up to some maximum length.
- i.e., just as we have an embedding for token *fish*, we'll have an embedding for position 3 and position 17.
- As with word embeddings, these position embeddings are learned along with other parameters during training.

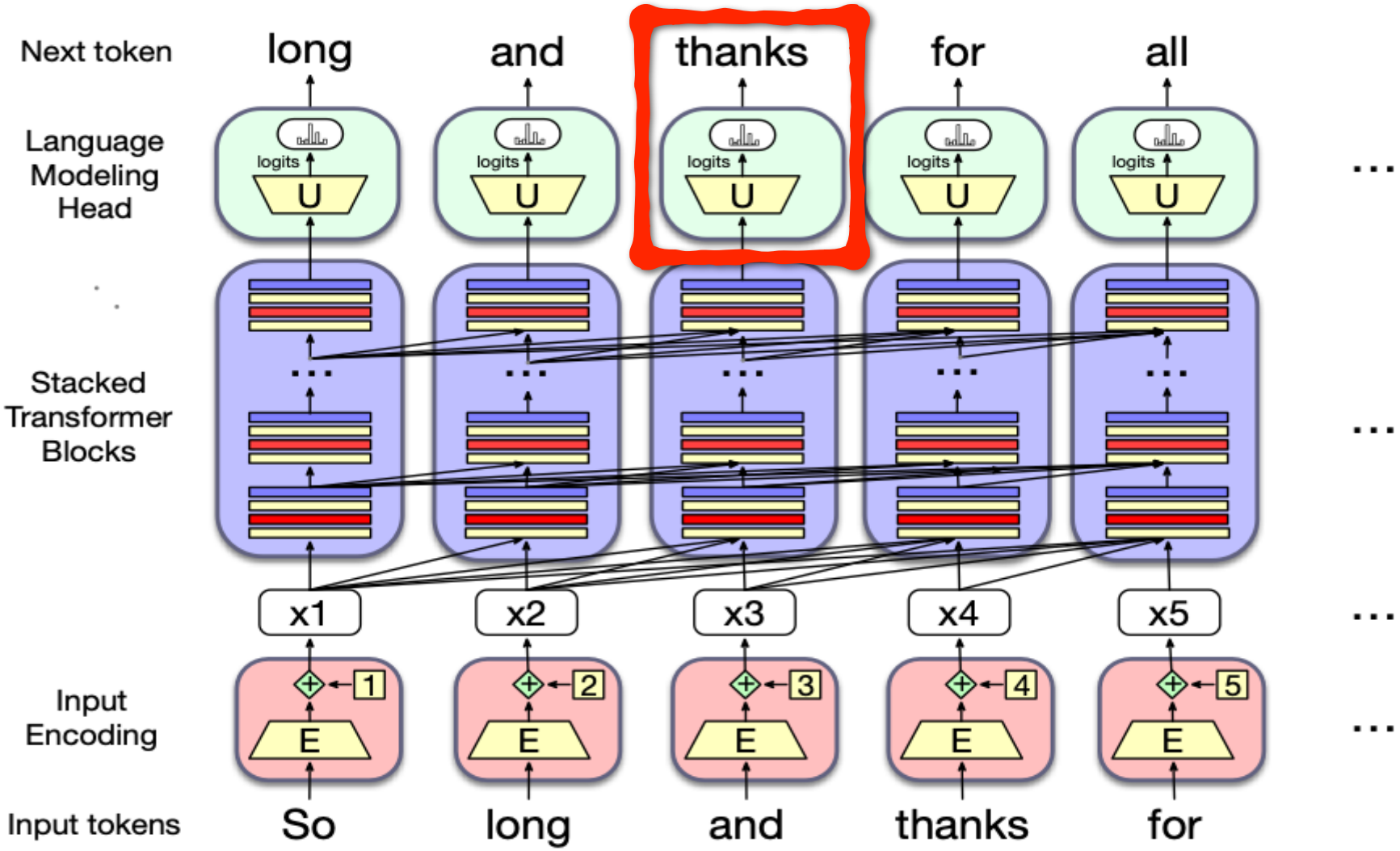
Each $\mathbf{x}_i$ is just the sum of word and position embeddings



Reminder: transformer architecture



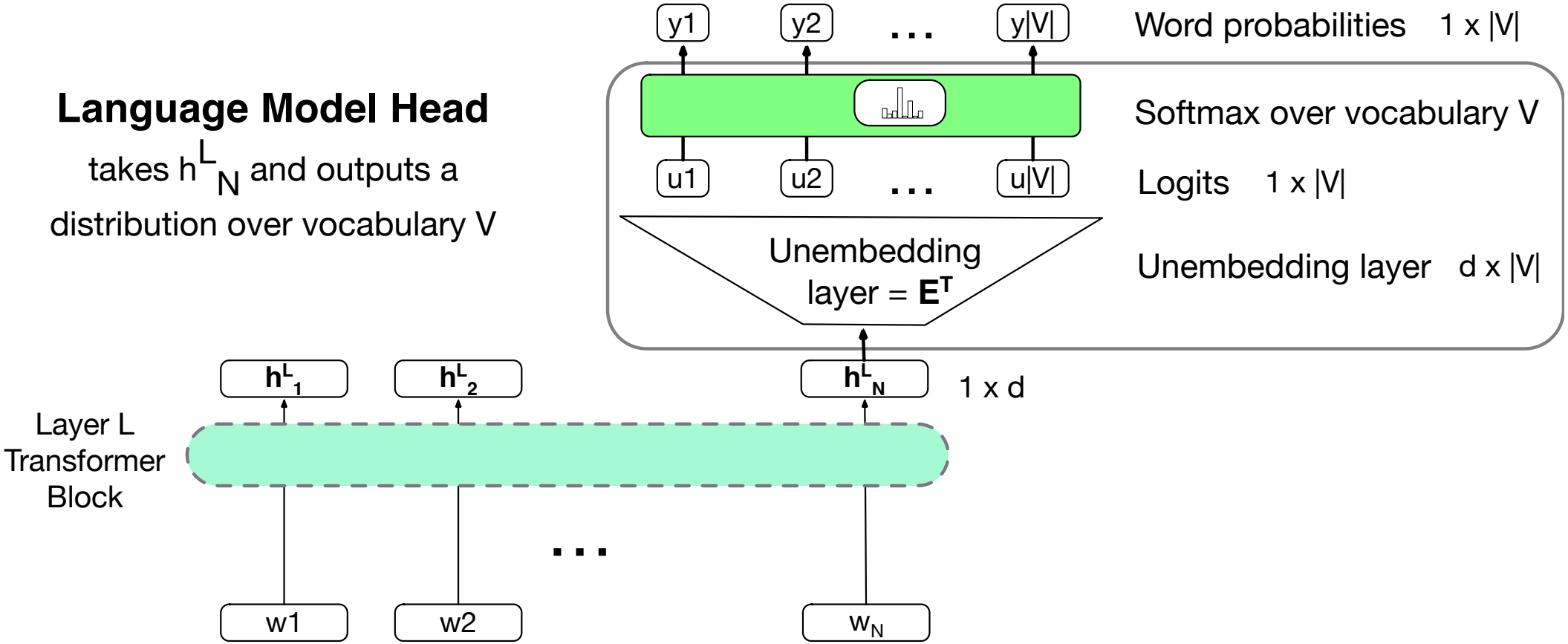
Reminder: transformer architecture



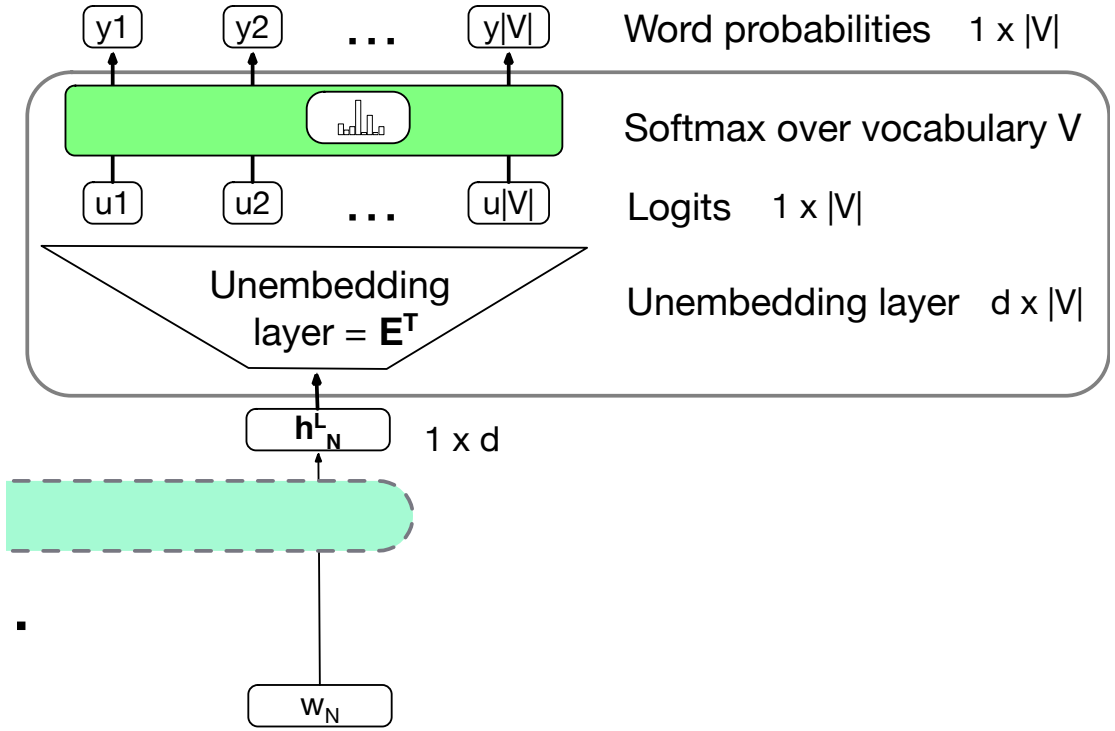
Language modeling head

Language Model Head

takes h^L_N and outputs a distribution over vocabulary V

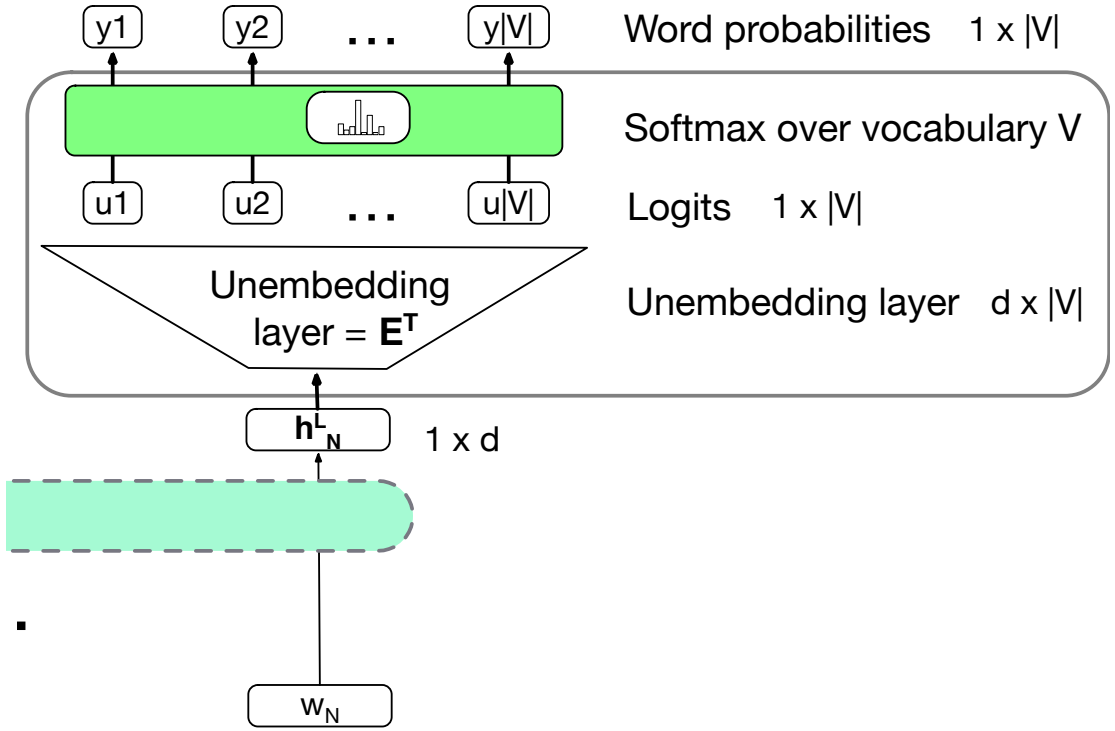


Language modeling head



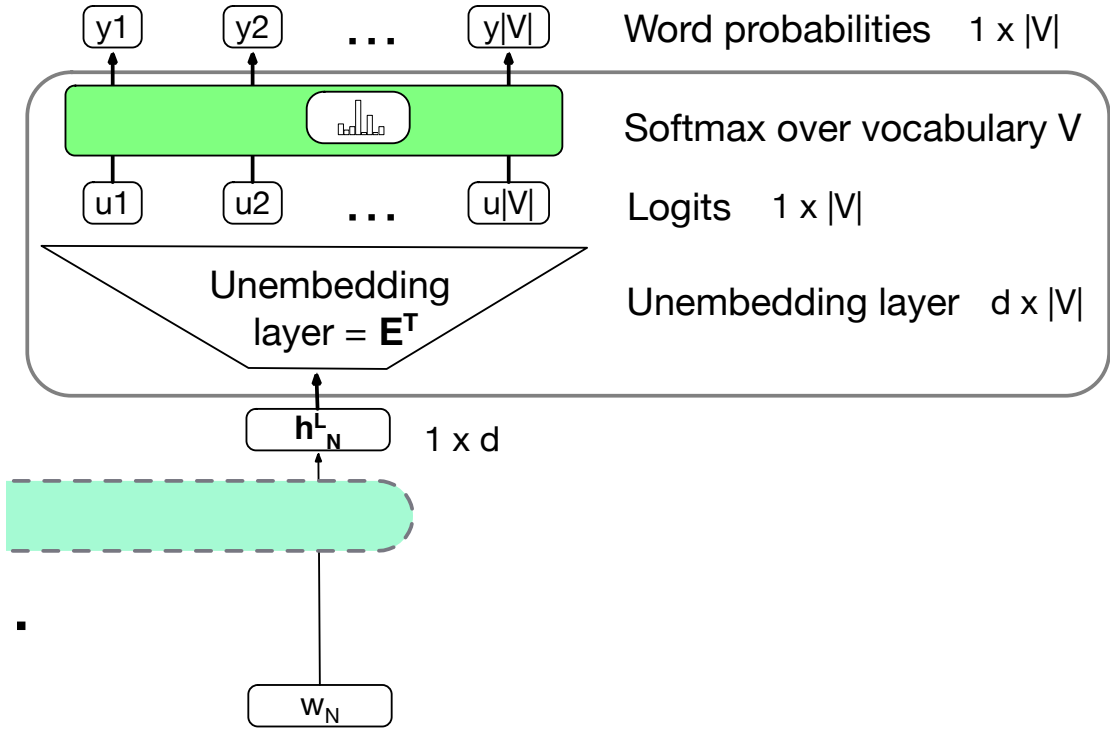
Language modeling head

Unembedding layer: linear layer projects from $\mathbf{h}_N^L$ (shape $1 \times d$) to logit vector



Language modeling head

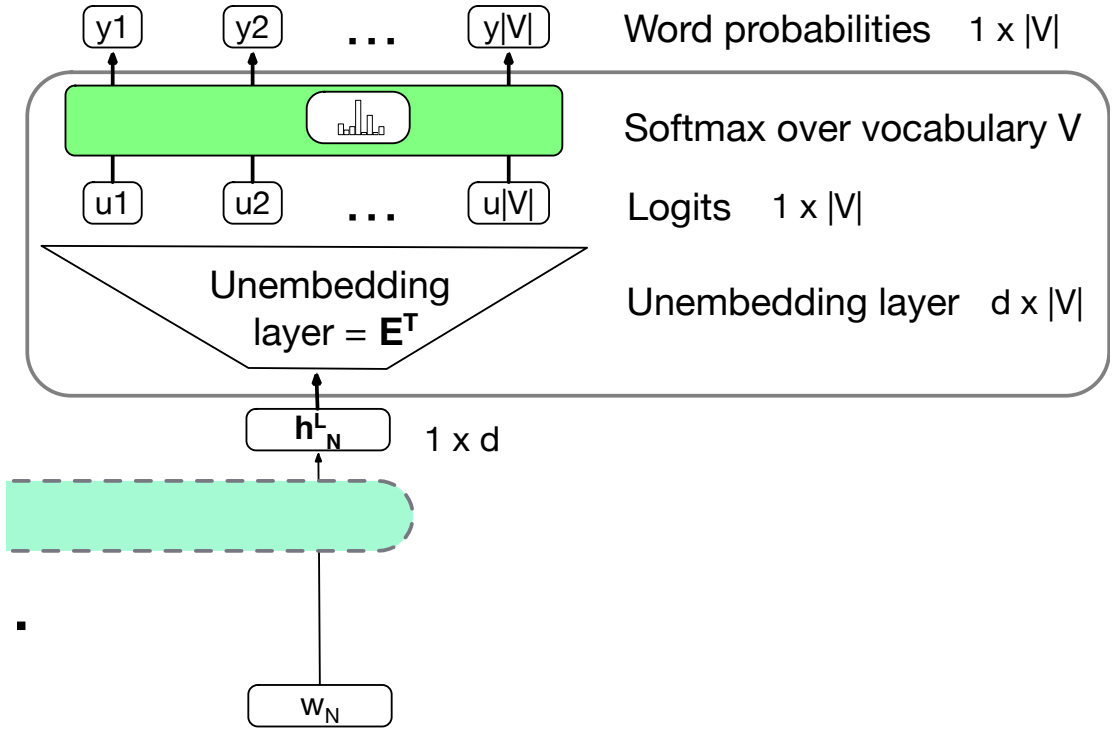
Unembedding layer: linear layer projects from $\mathbf{h}_N^L$ (shape $1 \times d$) to logit vector



Why "unembedding"? **Tied** to $\mathbf{E}^T$

Language modeling head

Unembedding layer: linear layer projects from $\mathbf{h}_N^L$ (shape $1 \times d$) to logit vector

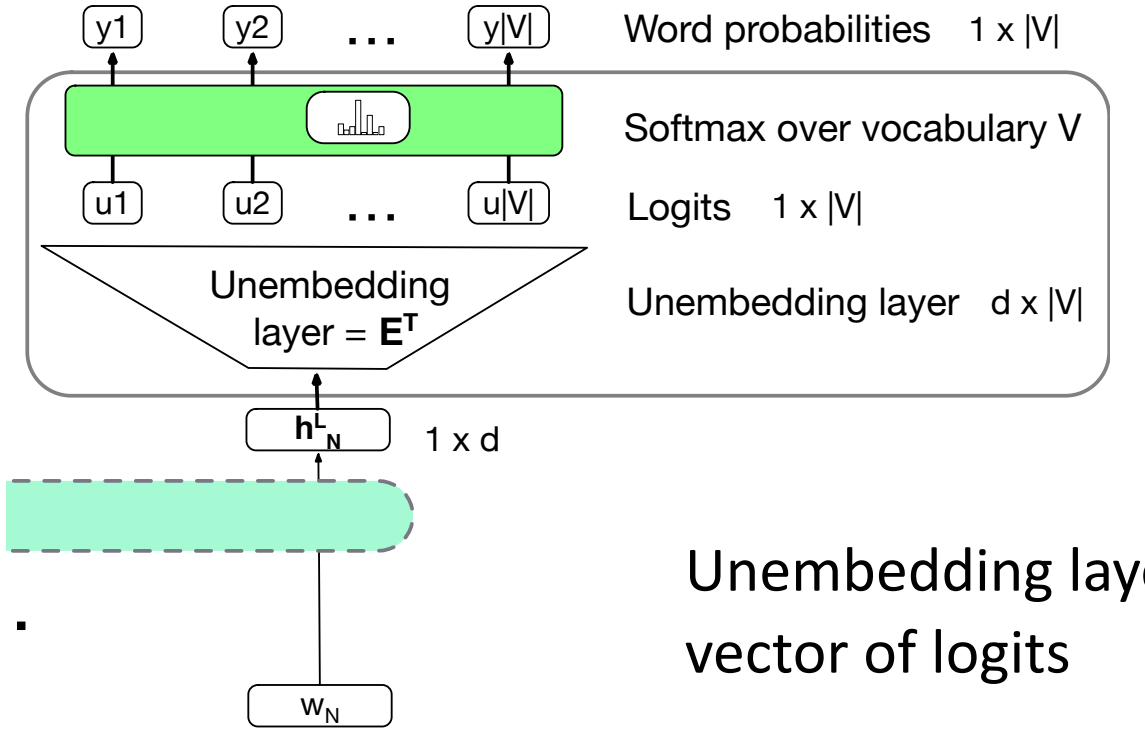


Why "unembedding"? **Tied** to $\mathbf{E}^T$

Weight tying, we use the same weights for two different matrices

Language modeling head

Unembedding layer: linear layer projects from $\mathbf{h}_N^L$ (shape $1 \times d$) to logit vector

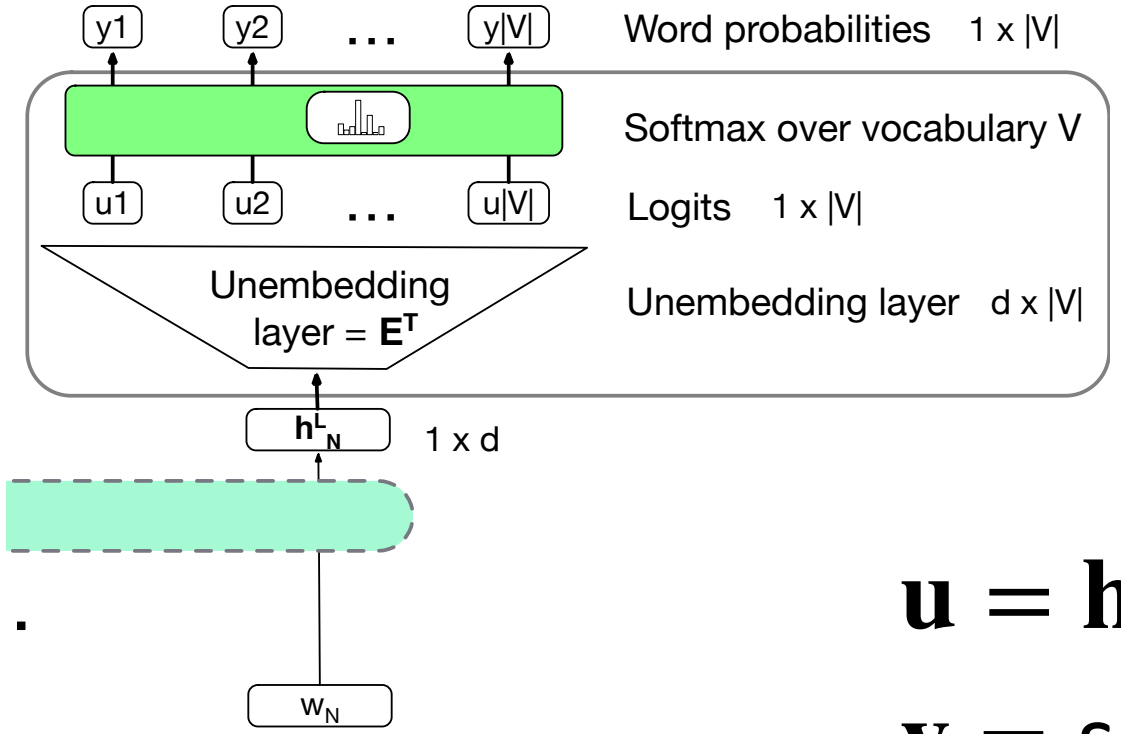


Why "unembedding"? **Tied** to $\mathbf{E}^T$

Weight tying, we use the same weights for two different matrices

Unembedding layer maps from an embedding to a $1 \times |\mathscr{V}|$ vector of logits

Language modeling head



Logits, the score vector $\mathbf{u}$, Shape $1 \times |\mathscr{V}|$.

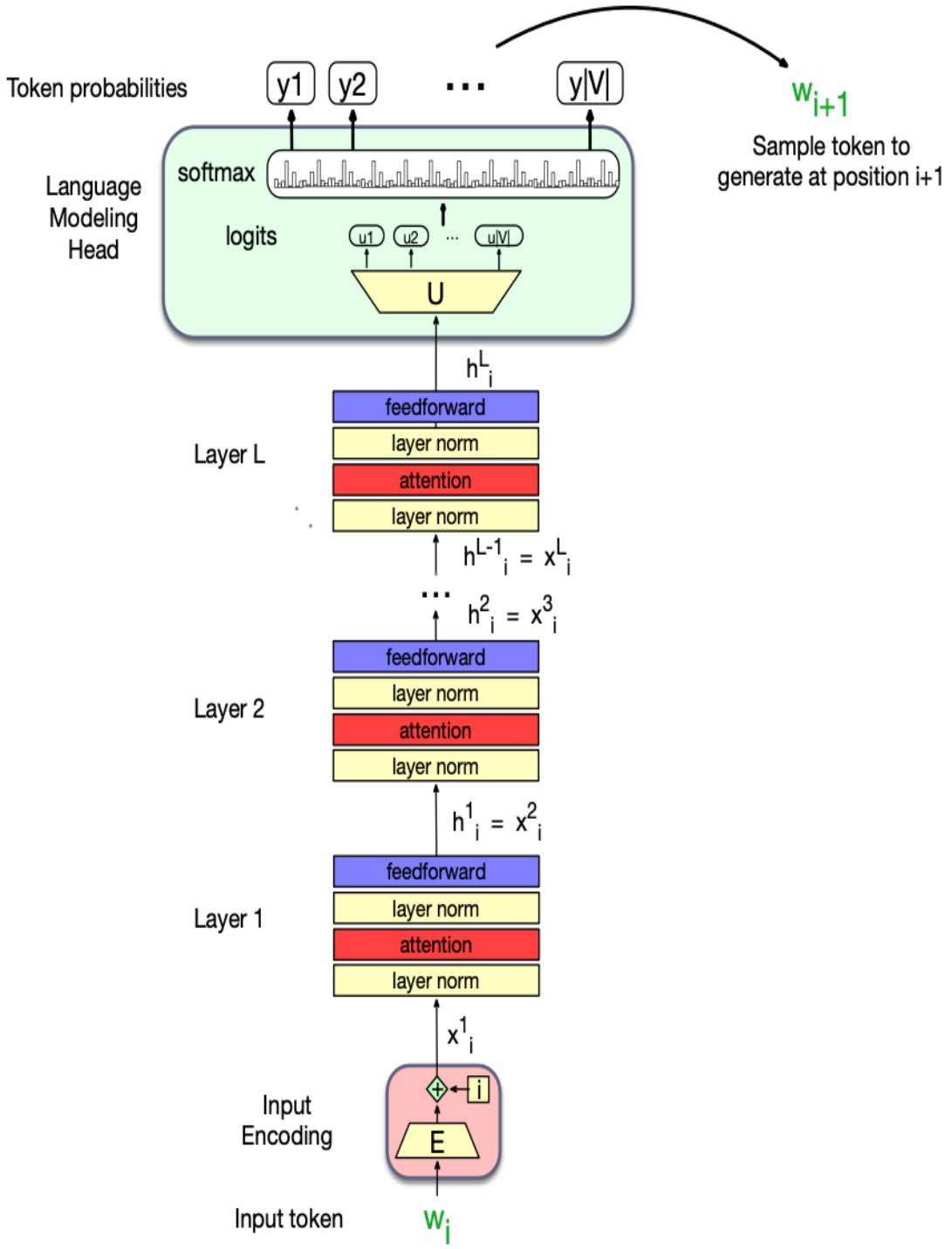
One score for each of the $|\mathscr{V}|$ possible words in the vocabulary $\mathscr{V}$.

Softmax turns the logits into probabilities over vocabulary. Shape $1 \times |\mathscr{V}|$.

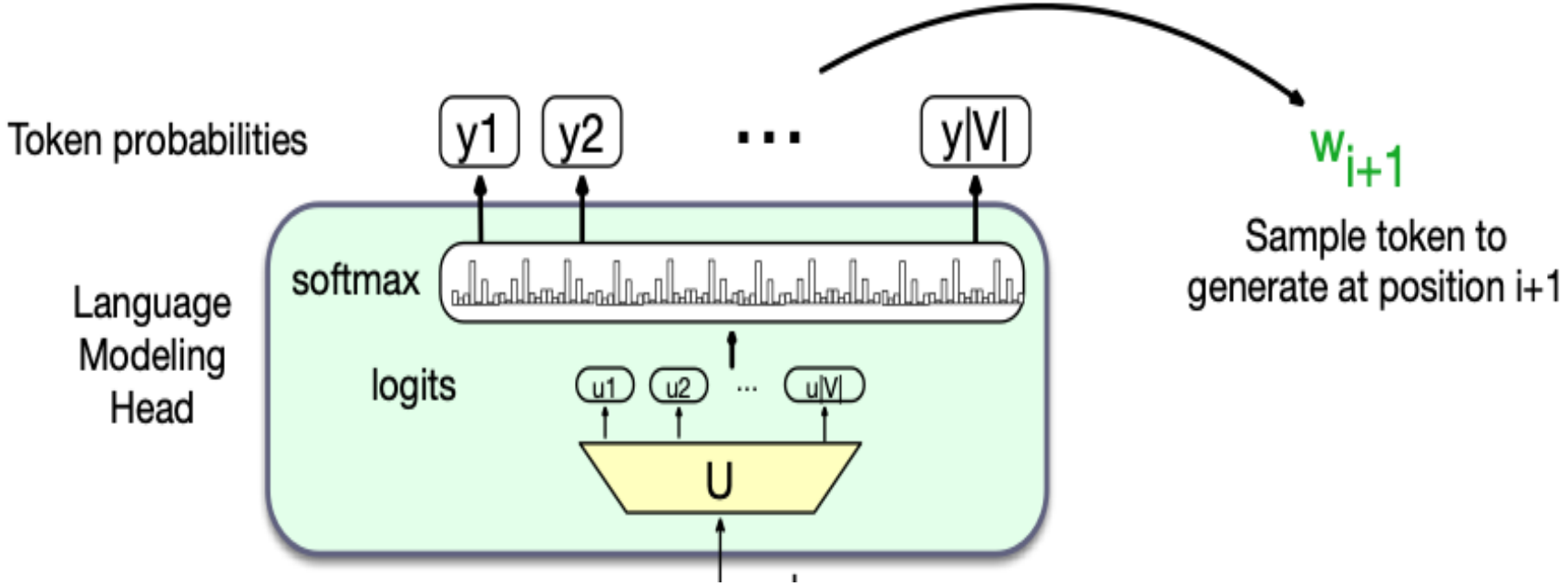
$$\mathbf{u} = \mathbf{h}_N^L \mathbf{E}^\top$$

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$

The final transformer model



The final transformer model



[15 minute break]

Working with Transformers!

Team up!

Open exercises/week 7 in your course folder and start writing/running code!